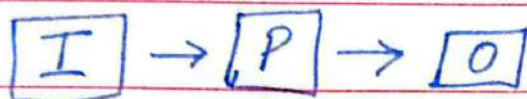


~ (Lecture no: 01) ~

Data Structures :-

• Algorithms :-

Set of instructions to solve a problem.



↓
basic operations are common in algorithms.

↳ Data reading.

↳ Data writing.

• (Storage & retrieving) •

→ Time & Space are the most important factors. (User ko faida hai).

• Data Structure deals with storage & organization of data in order to facilitate access and modifications.

↳ At times, ideal solutions are compromised.

Time {	most depreciated	Time {	Time X Space ✓
int hr; 0~23	5	unsigned char hr;	
int m; 0~59	6	// m;	
int s; 0~59	6	// s;	
}	(176)	}	(3B)
	↓ (12 B) ideal solution		↓ typecasted to int when displaying.

Time {	Time ✓ Space X	Time {	Accuracy issue ✓
int seconds;		double tm;	
}	(4B)	}	(8B)

→ is Symmetric in $O(1)$ space complexity & $O(N^2)$ time complexity.

Date _____

Note: 0 can also be input
for algorithms. Day
(implicit input)
others are explicit input.

Properties of Algorithms:-

- 1° Correctness.
- 2° Input/Output.
- 3° Definiteness. (Isn't vague) (Not ambiguous)
- 4° Feasibility.
- 5° Finiteness.

◦ Proofing Techniques of Discrete Mathematics.

◦ Testing is important.

↳ Algorithm Correctness.

↳ Problem Size. ◦ $f(N) = N^2$ (unit time).

↳ Problem instance. ◦ Sample Case.

(Lecture no: 02)

↳ {

≡

f()

≡

}

• void displayArray (int a[], int N)
{

for (int i=0; i < N; i++) N+1
{

cout << a[i]; N

}

}

Total = 2N+1
cost

↳ In LinearSearch the worst case is that value isn't found.

↳ We always have to check on worst case.

◦ while (low $\leq$ high) $\circ \log_2 n + 2$
 {

// all will have $\log_2 n + 1$

}

Total Cost = $5 \log_2 \text{size} + \frac{1}{9}$

◦ for (int i = N; i $\geq$ 0; i--) $\circ N + 2$.

◦ for (int i = 1; i * i $\leq$ N; i++) $\circ \sqrt{N}$

$$\hookrightarrow i * i \leq N$$

$$i^2 \leq N$$

$$i \leq \sqrt{N}$$

◦ Nested Loop Total Cost = $2N^2 + 2N + 1$

$\hookrightarrow$ for (i = 1, i $\leq$ N, i++) $\circ N$
 {

for (int j = 1; j $\leq$ i; j++)
 {
 }
 }

$$\circ \text{Total Cost} = \frac{1}{2}(N^2 + N)$$

Date _____

Day _____

- Selection Sort. $\rightarrow$ Time Equation = Quadratic
- Bubble Sort. $\rightarrow$ Time Equation = Quadratic
- $\rightarrow$ Do practice.

$\rightarrow$ Selection Sort:-

i	j
0	$\longrightarrow 1-N = N$
1	$\longrightarrow 2-N = N-1$
2	$\longrightarrow 3-N = N-2$
3	
⋮	
⋮	
⋮	
⋮	
$N-2$	$\longrightarrow N-1-N = 2$

$\rightarrow O(-)$ This is the dominant term.

• Space Equation:-

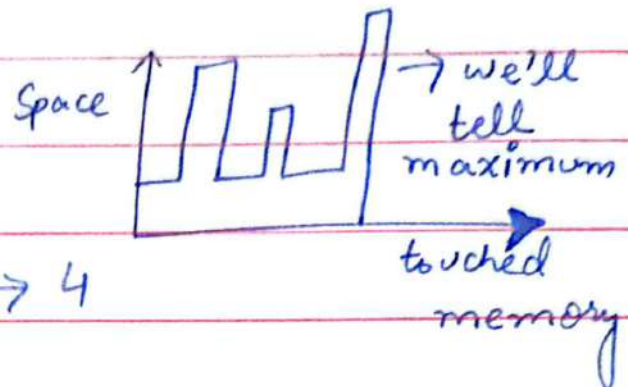
```

f()
{
    int a; → 4
    ≡
    {
        int b; → 4
    }

    {
        int *p = new int[N] → 4N + 4
        delete p;
    }
}

```

Total = $4N + 4$



↳ Maximum kitna touch hui memory woh likhna hai.

↳ Example 1:-

```

int *ptr;
for (i = 0; i < N)
{
    int *p = new int[N];
    delete p;
}

```

Date _____

$$N \text{ by } 2 = *$$
$$N \text{ over } 2 = /$$

Day _____

↳ `data = new int * [row];`

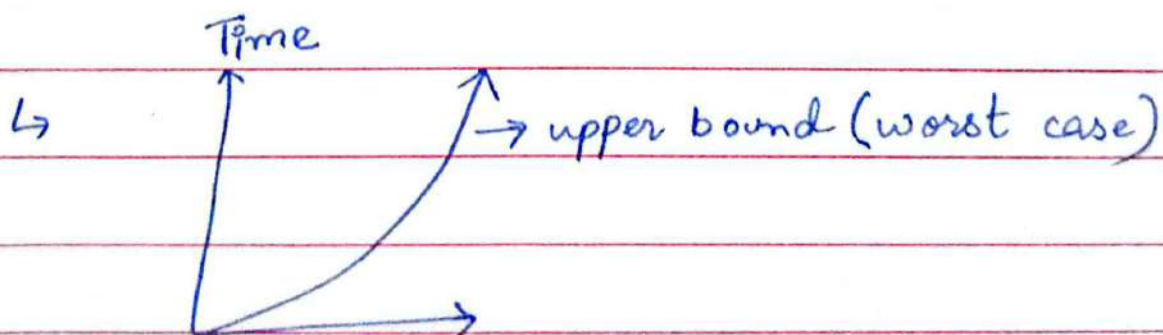
↓
agr language me
loop lagai hui
brnary k liye toh
row, abhi

ye assumption le rahay
1 unit time
lega. (Imp. point)

Example:-

$$\text{Cost} = N^2.$$

$$3N^2 + 30N + 100$$



$$0 \leq f(n) \leq cg(n) \rightarrow \text{for } O$$

$$0 \leq f(n) \leq cg(n) \rightarrow \text{for } \Omega$$

$$\hookrightarrow f(N) = O(n^2)$$

$$\hookrightarrow f(N) \neq O(g(n)) \downarrow$$

$\hookrightarrow f(N) \in O(g(n))$ $f(N)$ is big O of $g(n)$

$\hookrightarrow$ for Θ :-

$$C_1 g(n) \leq f(n) \leq C_2 g(n)$$

$\hookrightarrow$ iski itni upper & lower bound hai
(itna acha aur bure hai)

$\hookrightarrow$ worst aur best way ka Θ ho sakta but generically bat nhi kr saktay hum.

$\hookrightarrow$ exponential functions are not feasible.

$\hookrightarrow$ Equation does matter. (Analysis Example)

Date _____

Day _____

↳ we took N^2 rather than N^5 due
to the reason of tight bound.

Lecture no: 04

↳ Input space complexity me add nhi krtay agr output array input me orahi toh at the end woh hai toh output ka he hissa.

* $\log_b x = \frac{\log_a x}{\log_a b} \rightarrow \text{important property.}$

↳ $O(\log n) \rightarrow$ base nhi likhtay 0 me.
 ↳ sirf nature jan nay k liye $\log n$ likhte 0 me.

↳ we also consider base as constant.

→ $3 \log n + 1$

↳ $^3 O(\log n).$

↳ Dominant Term = $\log_{\frac{1}{3}} n.$

↳ Time measurement using chrono library.
 ↓
 Time in nanoseconds.

↳ timepoint $<$ $>$
↓
3 types by Sir. $\leftarrow$ System Clock.
Steady Clock.
High Resolution Clock.

↳ most compilers are doing high-resolution-clock in place of steady clock.

↓
compilers mostly doesn't implement it.

- accurate (more ticks)
- more affected by background processing

- consistent but less ticks.
- less affected by background processing.

↳ if implemented, then high-resolution-clock should be used.

↳ Cache also plays an important role in performance measurement.

↳ Cache in Binary Search (X)

↳ Cache in Linear Search (✓)

Date _____

Day _____

↳ Cache in Bubble Sort (X)

↳ Cache in Selection Sort (✓)

↑ most of the time.

↳ Average Case Analysis. (har jagah use hota)

↳ we use Θ in average case.

↳ Kabhi bhi accurate measurement

nhi ayegi, average sirf background processes ko nullify krega. Cache waghera ko nhi and mukhtalif aur factors ko.

$$\Delta \quad \frac{1}{t} = \frac{100}{0.5} = 200$$

$$\frac{100}{0.5} = \frac{1}{60 \times 60}$$

↳ $T(n) = C \cdot n$ → if quadratic or cubic just change n.

$$0.5 = C \cdot 100 \Rightarrow C = 0.005 \text{ ms}$$

↳ 1 min

$$\frac{1 \times 60 \times 1000}{0.005} = n$$

$$n = 12000000 \text{ ms}$$

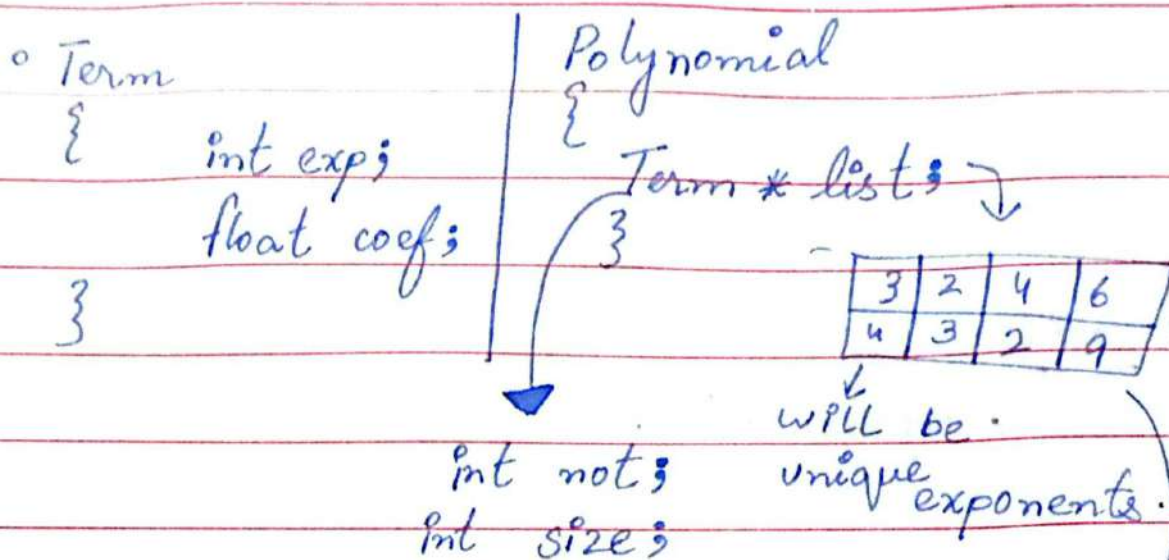
Date _____

~~37~~ + ~~1~~ ~~x~~

Day _____

4

↳ Make Term class for Scenario - ²/₄



↳ for clear() just use not=0;

→ $O(x+y)$

maintaining ~~it~~ it sorted!

↳ Merge sort is done in class. (+ operator)

↳

x^3	x^5	x^{10}	x^{50}	x^{80}
-------	-------	----------	----------	----------

$3 + 2 + 5 + 40 + 30 = 80$ exactly 0 times.

(Lecture no 05)

↳ Algorithms language se bias hokr nhi likhnay.

↳ Think More, Write Less. (Most Imp.)

Arrays:-

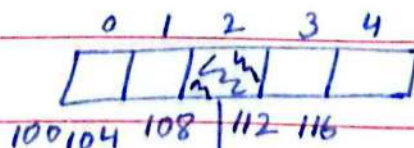
- ↳ representation. (In Today's Lecture)
- Consecutive Memory Location of same Size.
- Direct Access / Random Access.
 - ↳ har location pr jane ki cost same (constant) hai.

↳ Search → location of data isn't known.

↳ Access → location of data is known.

Sir's Explanation:-

`int a[5];`



↳ `float *p = (float *) &a[2];`

1D Array Mapping:-

$1D(i) \rightarrow \text{index of location.}$

$$\hookrightarrow \text{address} = \text{baseAddress} + (\text{index} * \text{sizeOfEach Location})$$

$\rightarrow$ Doesn't matter the size of 1D Array to go ~~from~~ to any location constant time is required. (Hence Proved)

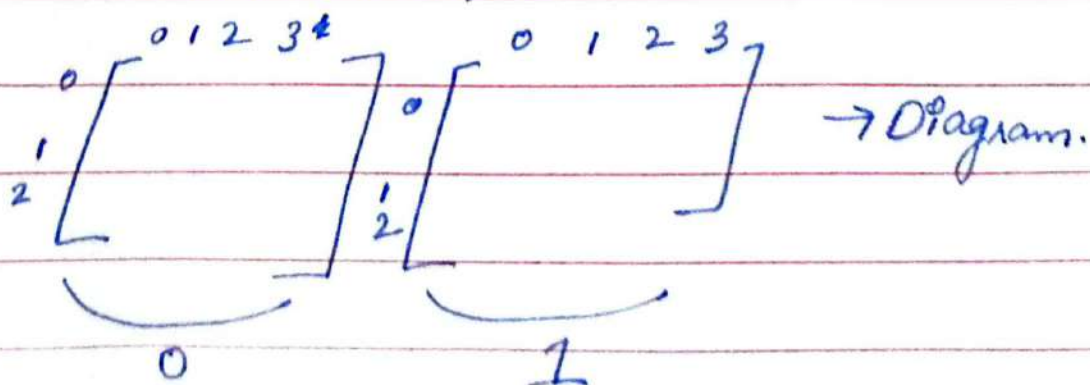
2D Array Mapping:-

$$\hookrightarrow \text{index} = i * \text{columnSize} + j.$$

$\hookrightarrow$ ND Array Mapping $\left\{ \begin{array}{l} \text{C++ (Row M.)} \\ \text{(Column M.) Matlab.} \\ \text{(Liffee Vector) C\#} \end{array} \right.$

3D Array Mapping:-

$\rightarrow \text{int } a[2][3][4]$



⇒ Note:- Logic language independent ~~honi~~ chahiye.

Date _____

Day _____

$D_1/x \quad D_2/y \quad D_3/z$

* `int a[2][3][4];`

`a[i][j][k]`

$$\hookrightarrow 3D(i, j, k) = i * yz + j * z + k.$$

ND Array Formula / Mapping:-

$$ND(i_1, i_2, i_3, \dots, i_n) = \sum_{p=1}^N \left(i_p \prod_{q=p+1}^N D_q \right)$$

↳ Agar array ka index 1 se start hoga toh bus thora offset $\pm$ hoga koi major change nhi ayega.

↳ baqi languages me bound checking hai mapping se related.

↳ ND array mapping formula says that 2D, 3D, 7D etc me different time lag rah.

↳ Practically 10D, 15D use nhi krta hai.

↳ As a comparison 3D and 4D me cost ka difference negligible hai.

❖ Column Major Order:-

$$2D(i, j) = j * \text{rowSize} + i.$$

Generalized Formula:-

$$ND(i_1, i_2, \dots, i_n) = \sum_{p=1}^N \left(i_p \prod_{q=1}^{p-1} D_q \right)$$

'OR'

$$\sum_{p=1}^N \left(i_p \prod_{q=1}^{p-1} D_q \right)$$

↳ In C++, row wise will perform better because of maximum cache performance.

↳ In ~~matlab~~ matlab, columnwise will perform better.

→ jagged arrays can have different sized rows.

↳ Jagged Array ki availability k chances zyada hain.

* Column Major for 3-D Array.

~ Lecture no 06 ~

◦ Sparse Matrices:-

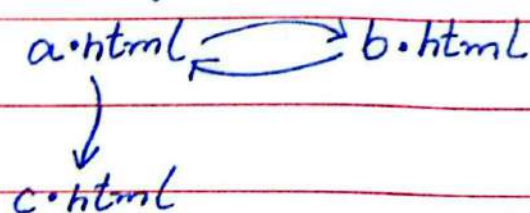
↳ Web Connectivity Graph.

→ Go from one page to another.

$$\begin{matrix} & \begin{matrix} a & b & c \end{matrix} \\ \begin{matrix} a \\ b \\ c \end{matrix} & \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

◦ Graph

◦ Diagram.



↳ Enhances (Memory + Time) Cost.

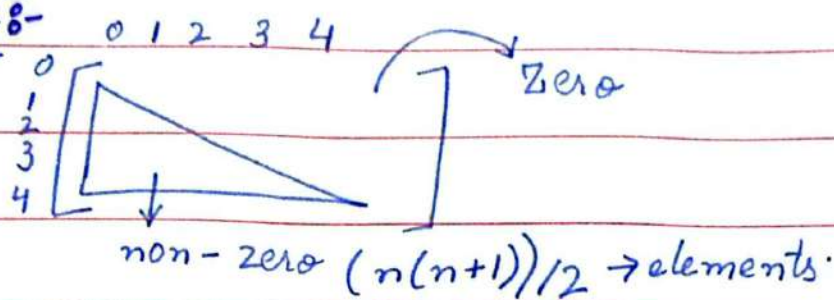
↳ Term Polynomial Class is Sparse Polynomial and the other is Dense Polynomial.

↳ We'll draw mapping schemes for small sparse matrices.

◦ Types of Sparse Matrices:-

- ↳ Having a pattern.
- ↳ Not having a pattern.

◦ Example ^①:-



→ Memory Saved: $N^2 - (N(N+1))/2$

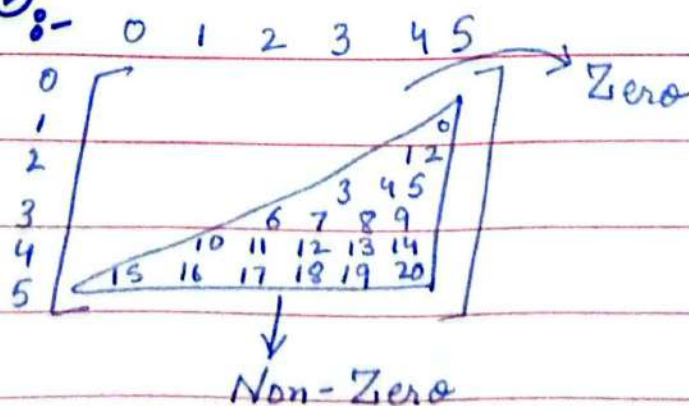
◦ Mapping:-

$$\text{map}(i, j) = \frac{i(i+1)}{2} + j$$

→ applicable when $(i < j)$

↳ Constant time Mapping for the above Matrix.

◦ Example ^②:-



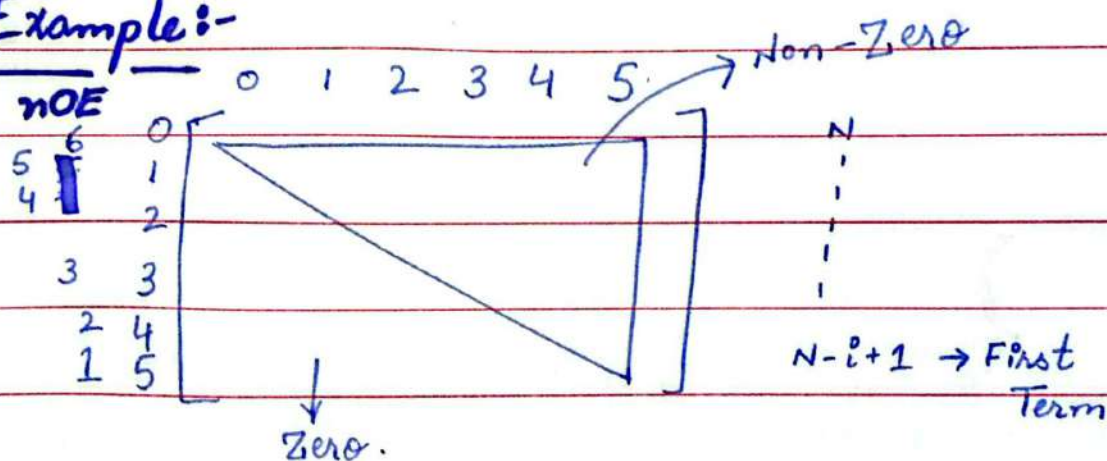
→ Saved Memory :- $N^2 - \frac{N(N+1)}{2}$

◦ Mapping :-

$$\text{map}(i, j) = \frac{i(i+1)}{2} + (N - (i+1))$$

↳ Only applicable when (i, j) is given and $j \geq (N - (i+1))$

③ ◦ Example :-



$$\text{No. of Terms} = \text{Upper Bound} - \text{Lower Bound} + 1$$

$$i = \text{no. of Terms}$$

① Formula = $\frac{i}{2} (2N - i + 1)$

② $N - i$

1

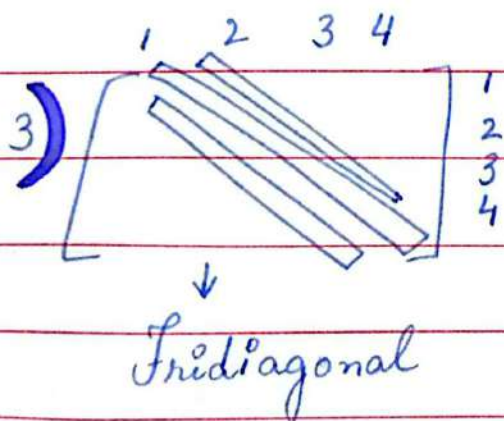
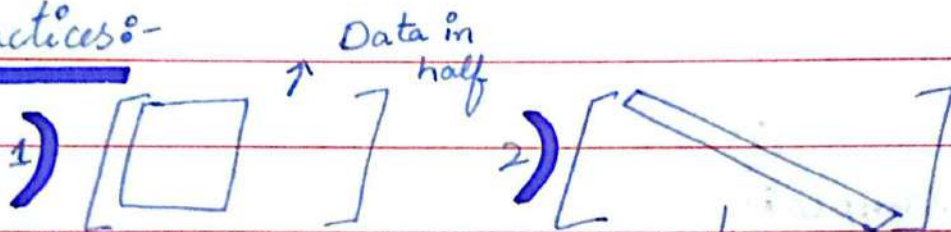
no. of Terms = $N - i$

$$\rightarrow \frac{n}{2} (2a_1 + (n-1)d)$$

$$\Rightarrow \frac{(N-i)(N-i+1)}{2} \rightarrow \text{2nd Formula}$$

↳ Only applicable when $(i \geq j)$

↳ Practices:-



→ Generalize
Form (α β Band
Matrix)

α = upper Diagonals
 β = Lower Diagonals

↳ Not Having a Pattern:-

↳ We'll store
value, index and row pointer (
starting of that row)

Date _____

Containers → which can contain something.
Iterators → Iterate over something.

(Lecture: nos 07)

Stacks:-

- ↳ LIFO (Last in first out) Principle.
- ↳ Any container that exhibits LIFO behaviour then we can call it a stack.

Note:-

- Insert = push
- Remove = pop

cout ko jab
char * milega woh
tab tk display krega
jab tk '0' nhi milega.

Impo Discussion:-

int a = 10;

int ~~11~~ b = 90;

↑
a-10
9
8
7
b-6

◦ Little Endian
= Lower B, Lower A
= High B, High A

Memory

allotment

Guarantee
nhi k aik dum nee chahoga but ye guarantee k kam
he hoga.

- ↳ unsigned long long int me typecast
krein toh 64th bit (minus/2's complement)
wala issue nhi hoga.

↳ Heap me koi tarteeb nhi hai.

→ f(String a, String b)

↳ guarantee nhi kon pehle
banay ga (Compiler dep.)

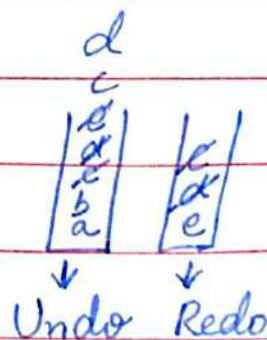
↳ When a function calls another function the called function's variable will be active and rest will be inactive that are outside the function.

↳ C++ me Assembly code generate hota (check krna hai).

↳ Alias ka size include nhi krna (backend pr pointer banata), alias problem size k according nhi ban sakte so constant he rehaye ga.

Examples of Stack:-

↳ Undo/Redo.



- Agar new operation krte Undo k baad toh baki Redo khtm hojatay hain.

Date _____

* Ho sakta k aik algorithm kuch problem instances pr worst ho aur baki pr normal so wki average kafi achhi ho.

Day _____

↳ Expression Evaluation.

↳ Backtracking.

Stack ADT:-

↳ $top = numberOfElements$.

↳ refers where next insertion is
People also start to be done
from -1 in their implementation.

→ if $top == capacity$ (Full)

→ if $top == 0$ (Empty)

→ Amortized Analysis

In this we see multiple cases. ↓ push graph

Diagram:-



↳ We can't change worst case but
we can ~~change~~ average cost.
change

Date _____

Day _____

↳ Agar resize double krte hain
toh average cost 1 ayege.

Lecture no: 08

- Expression Evaluation.
- Rat in a Maze.

Expression Evaluation :-

- ↳ Infix^x $(a+b)$
- ↳ Postfix $(ab+)$
- ↳ Prefix $(+ab)$

Note:- We are assuming that our (operators)^x operands are unary.

- ↳ The most higher precedence is solved first.

Conversion from Infix ~~to~~ to Postfix/Prefix:-

- | | |
|--|--|
| <p>①</p> <p>↳ $a+b*c$</p> <p style="padding-left: 40px;">→ $a+bc*$</p> <p style="padding-left: 40px;">→ $abc*+$</p> | <p>②</p> <p>↳ $a*b+c$</p> <p style="padding-left: 40px;">→ $ab*+c$</p> <p style="padding-left: 40px;">→ $ab*c+$</p> |
|--|--|

- ③
- ↳ $a+b-(y+x*z)/T$
- $a+b-(y+xz*)/T$

$$\rightarrow a + b - \underline{y x z * + / T}$$

$$\rightarrow a + b - \underline{y x z * + T /}$$

$$\rightarrow \underline{a b + - y x z * + T /}$$

$$\rightarrow \underline{a b + y x z * + T / -} \quad (\text{Postfix form})$$

◦ Steps to solve:-

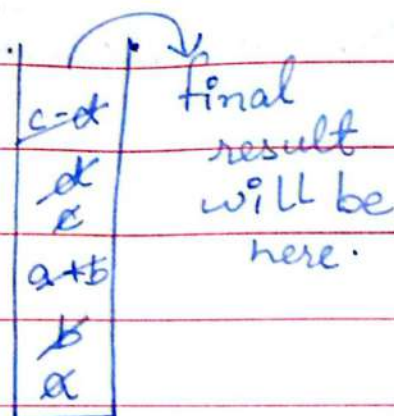
↳ push operands.

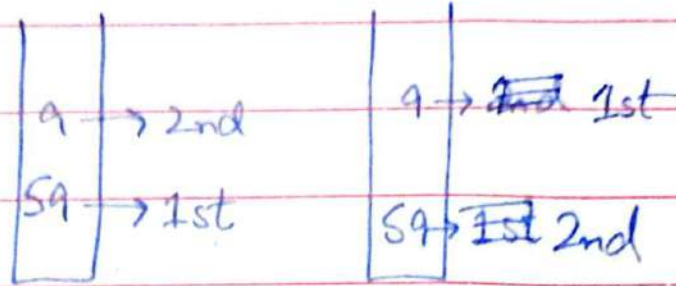
↳ if comes operator, pop last 2 and push the result to stack.

↳ At the end, result will be in stack.

↳ In prefix (R → L)
↳ In postfix (L → R)

Read from.





↳ So division and minus ka dhayan rakhna hai.

↳ 12 + 13

12, 13, + → we did tokenizing.

Conversion:-

∴ Stack of operators

• $a + b * c / d + e$

↳ push operands in String
& operator in Stack.

+
/
*
+

Date _____

Day _____

$$\hookrightarrow a - b * c \wedge (d - x + y / z) \wedge e + (f - g)$$

<u>Input</u>	<u>Stack</u>	<u>Output</u>
a	_____	a
-	_____	a
b	_____	ab
*	_____ *	ab
c	_____ *	abc
^	_____ * ^	abc
(	_____ * ^ (	abc
d	_____ * ^ (	abcd
-	_____ * ^ (-	abcd
x	_____ * ^ (-	abcdx
+	_____ * ^ (+	abcdx -
y	_____ * ^ (+	abcdx - y
/	_____ * ^ (+ /	abcdx - y
z	_____ * ^ (+ /	abcdx - yz
^	_____ * ^ (+ / ^	abcdx - yz
k	_____ * ^ (+ / ^	abcdx - yzk
)	_____ * ^	abcdx - yzk ^ / +
^	_____ * ^ ^	abcdx - yzk ^ / +
e	_____ * ^ ^	// e

Note:-

$$N! > 2^n$$

Date: _____

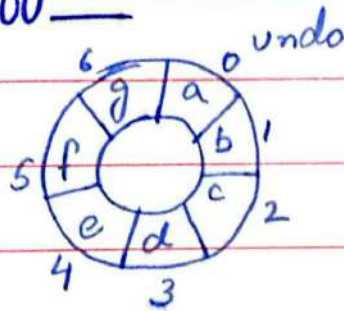
Day: _____

Lecture no: 09

$$\hookrightarrow \min = j; \rightarrow \frac{N(N-1)}{2} \quad \bigg/ \quad \frac{N^2}{4}$$

$\hookrightarrow$ String ki cost sirf push pop me add nhi krni.

Circular Buffer :-



$\hookrightarrow \text{top} = (\text{top} + 1) \% \text{capacity}; \rightarrow \text{for circular Buffer.}$

Queue :- (FCFS, FIFO, LILO)

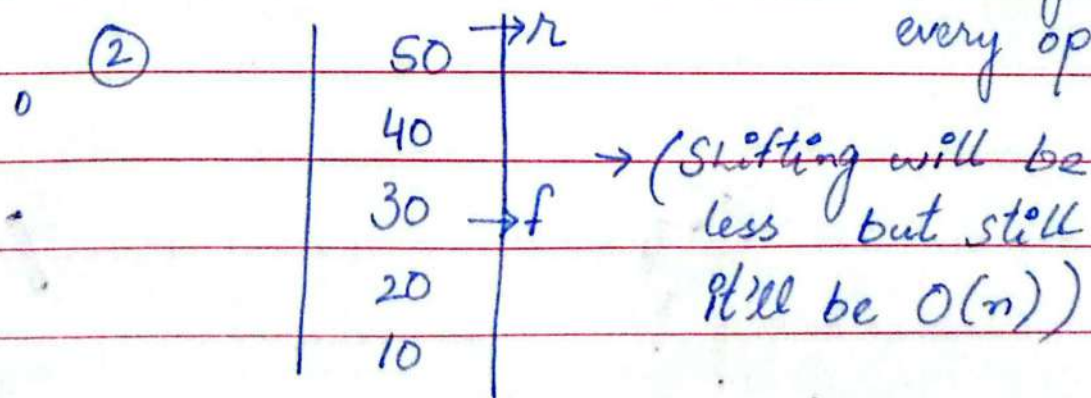
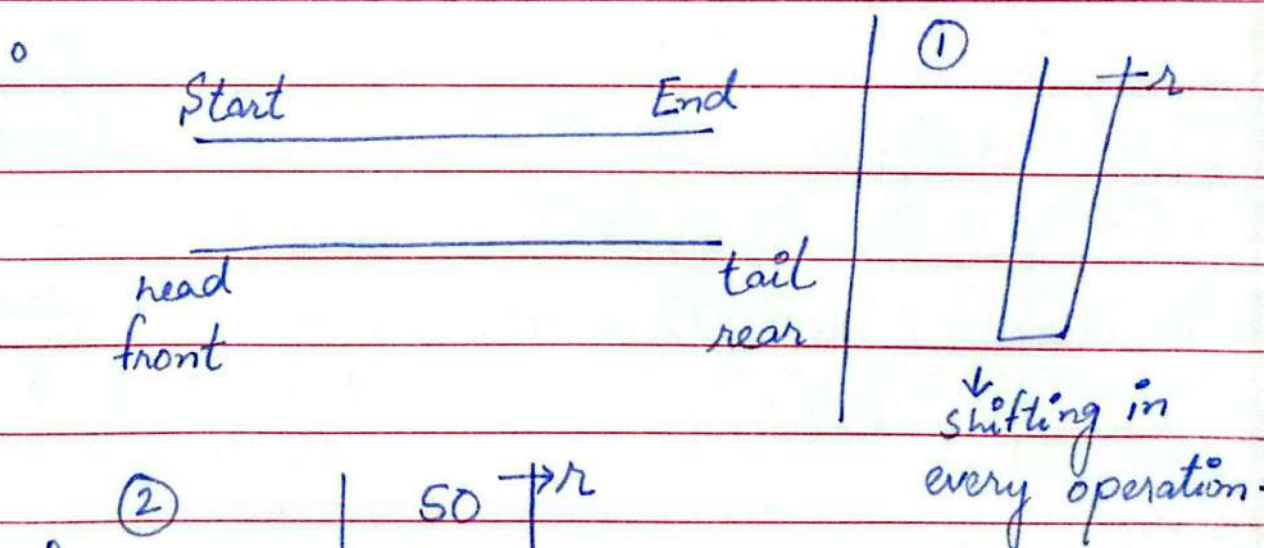
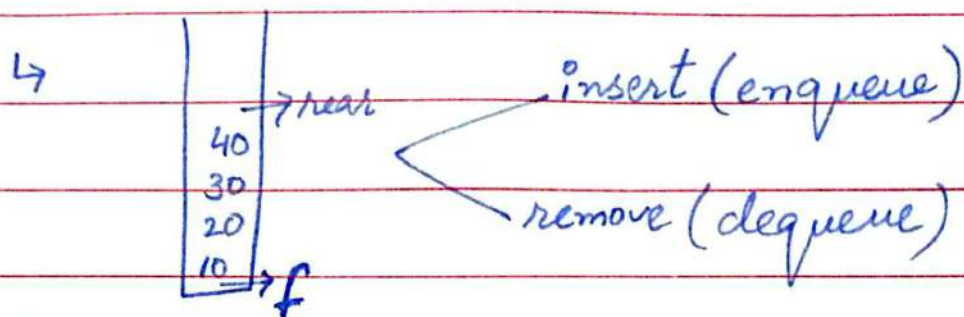
$\hookrightarrow$ Agar resources hon toh queue lagana ki zaroorat nhi hoti.

$\hookrightarrow$ Priority Queue.

↳ Primitive & Non-Primitive

↓
 foran kaam
 rok kr
 dusra kaam
 karta.

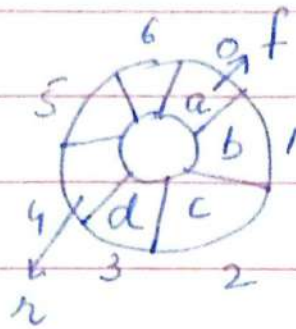
↓
 pichla kaam
 krke dusra
 kaam karta hai-



Date _____

Day _____

③



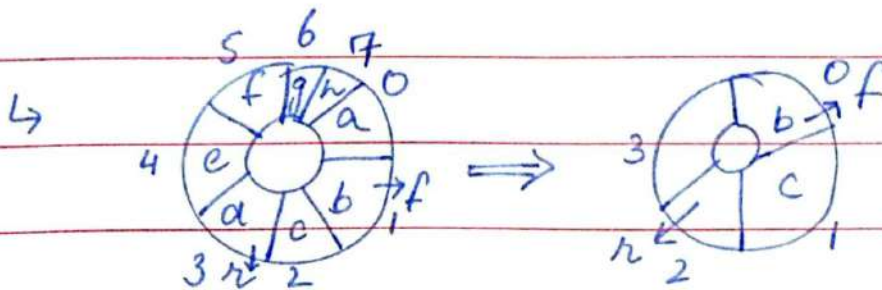
↳ Solution: take a variable no.

∴ $r == f$

↳ can be Full

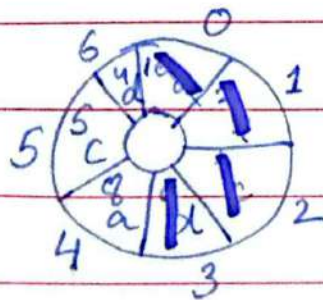
||
↳ can be Null.

↳ front se lekr $r-1$ tk data para hua hai-



↳ Double Ended Queue (Deck) → can be used as a stack & also as a queue.

↳ Operating System Working:-



∴ Round Robin / Time Sharing Algorithm.

Maze:-

	0	1	2	3	4	5	
0	0	0	0	0	1	0	
1	0	1	0	0	0	1	
2	0	0	1	1	0	1	
3	0	0	1	0	0	0	
4	0	0	0	1	1	0	

$(0,0) \rightarrow (4, \overset{1}{4})$

↳ 1 → block.

↳ 0 → Open.

↳ No Path: When stack empty and no available path to go.

(Lecture no: 10)

Recursion:-

↳ it means repetition.

1) Direct Recursion. (Self Call) $\left[\begin{array}{l} \text{void f()} \\ \{ \\ \quad \text{f()}; \\ \} \end{array} \right]$

2) Indirect Recursion. (Two functions

$\left[\begin{array}{cc} \text{f()} & \text{g()} \\ \{ & \{ \\ \quad \text{g()} & \quad \text{f()} \\ \} & \} \end{array} \right]$ calling each other)

↳ void f()
{

 cout << "1";

 f();

}

∴ Function call stack will overflow at some point when we'll call f().

• Termination case for recursion is called base case.

↳ void f(int N)
{

 if (N <= 0)

Note:- p = Print.
R = Return.

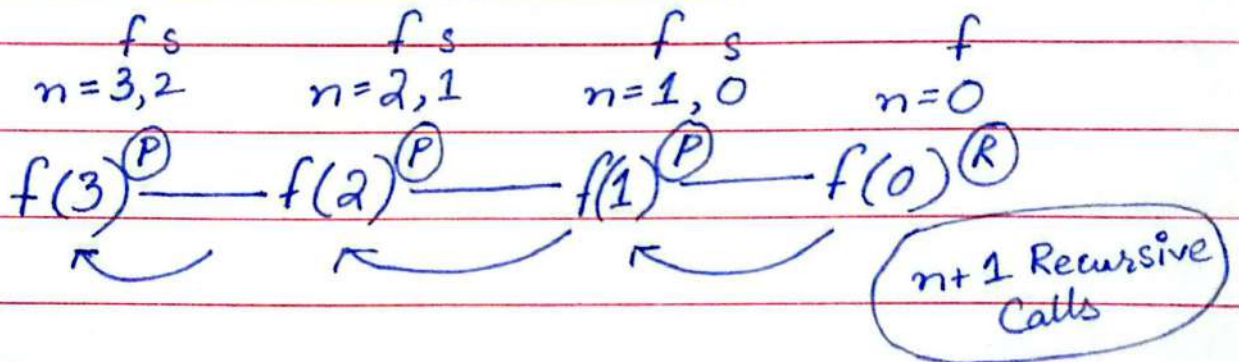
Date _____

Day _____

```
return;
cout << "1";
f(N-1);
}
```

Output:-
111

Function Call Tree / Stack :-



```
↳ void f(int N)
{
    if(N <= 0)
        return;
    cout << N;
    f(--N);
    cout << N;
}
```

Output:-
321012

↳ Above is its
function stack

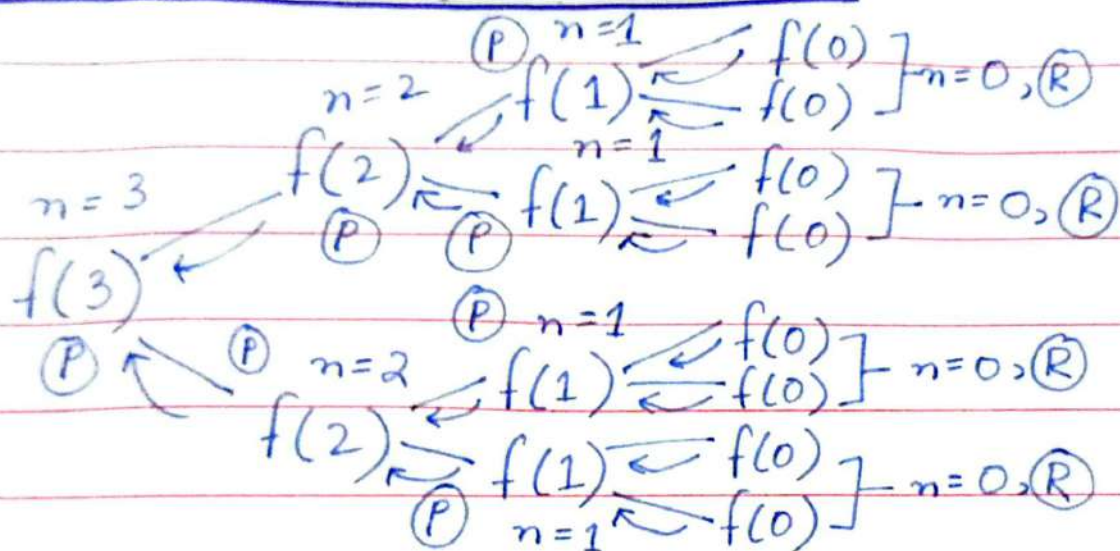
```
↳ void f(int N)
{
    if(N <= 0)
        return;
    cout << N;
```

$f(N-1);$
 $f(N-1);$
 $\}$

Output:-

3211211

Function Call Tree of Above:-



↳ Watch uper wali puri picture
 nhi dikha sakti hai sirf aik
 he function ki picture dikha
 sakti hai.

• How many recursive calls generated
 $= 2^{n+1} - 1$

• Above is $\times$ (the) $\times$ exponential function.
 an

- Recursive code is generally better when we're doing something while backtracking in recursive calls. (Hamasta aisa nhi hoga)

- ↳ Tail Recursion: When nothing is done while ^{backtracking} $\uparrow$.
- ↳ Body Recursion: When something is being done while backtracking.

```

↳ void f(int N)
{
    if (N <= 0)
        return 0;
    cout << N;
    f(N-1);
    N = N-2;
    f(N);
    cout << N;
}

```

Output:-

4321-101-1¹21-102

→ This is recursion's step count.

- Draw Trace for every code even if it's easy or hard.

```

↳ void print (int a[], int N)
{
    if (N <= 0)
        return 0;
    cout << a[N-1];
    print (a, N-1);
}

```

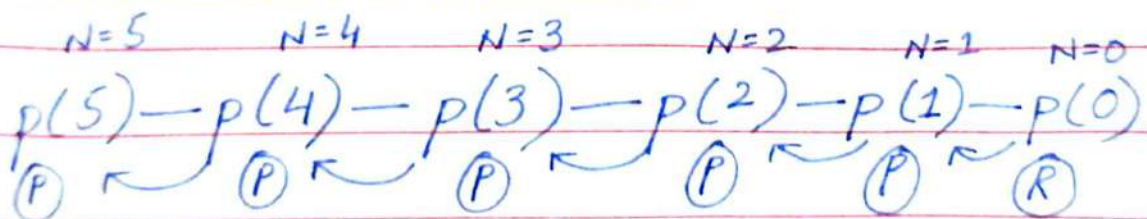
↳ $x[5] = \{1, 2, 3, 4, 5\}$

↳ `print(x, 5);`

Output:-

5, 4, 3, 2, 1

Recursive Trace of Above:-



```

↳ void print (const char* * p)
{

```

```

    if (*p == '\0')
        return;

```

```

    cout << *p;
    print (p+1);
}

```

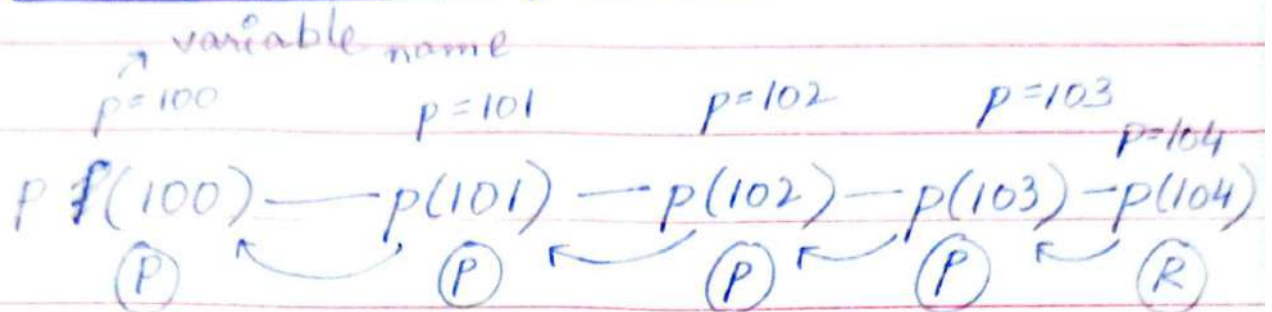
100 101 102 103 104

A	B	C	D	\0
---	---	---	---	----

Output :-

ABCD

Recursive Trace of Above:-



```

↳ void print (Stack <int> & s)
{

```

```

    if (s.isEmpty())
        return;

```

```

    cout << s.pop();
    print(s);
}

```

Output:-

40
30
20
10

40 30 20 10

Recursive Trace of Above:-

40
30
20
10

print(s) — print(s) — print(s) — print(s)

(P) (P) (P) (P)

 — print(s)

 (R)

o Jab by reference ho toh just aik
Single copy apni pass rakhti hai.

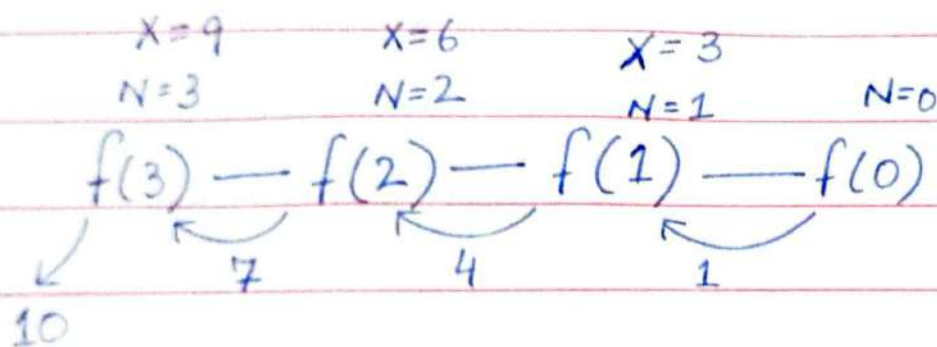
```

↳ int f(int N)
{
    if (N <= 0)
        return N+1;
    int x = f(N-1)+2;
    return x+1;
}

```

Final Returned
Value for $f(3)$:-
10

Recursive Trace of Above:-

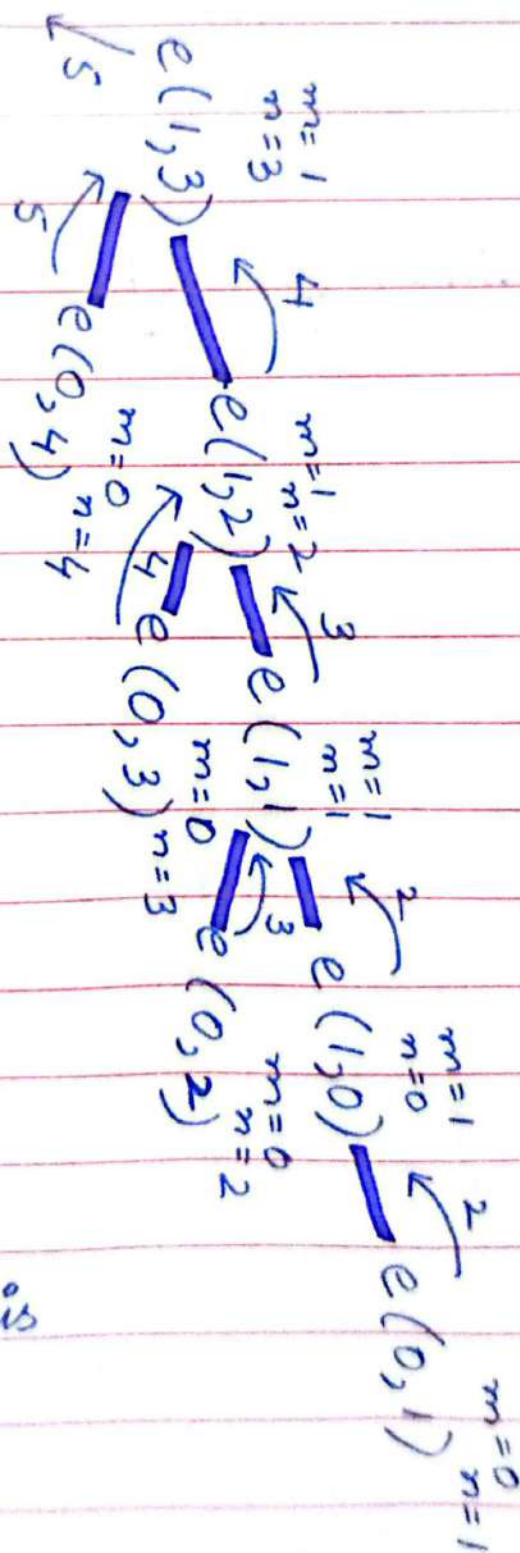


↳ Har code iterative likha ja sakta
but har iterative code purely
recursive nhi likha ja sakta hai. (Imp)

↳ Polynomial Time meaning n^1, n^2, n^3 etc

(Lecture : no: 11)

Trace for Enigma:-



∴ Must follow this pattern.

↳ In Quizzes and paper, we'll do trace but can do cut short if the picture while doing the steps first is ^{at time} ~~is~~ ^{is} ~~(de)~~ cleaner.

Recursive Linear Search :-

```
bool ls (int* arr, int index, int key)
{
    if (arr[index] == key)
        return true;
    if (index == 0)
        return false;
    return ls(arr, index+1, key);
}
```

∴ Workhorse

* Agar straight search krna hai toh size bhi pass krna paray ga.

→ If user wants better interface than.

```
bool is(int *arr, int N, int key)
{
    return is2(arr, N-1, key);
}
```

Interface

↳ Abhi humne valid inputs dekhni hain.

```
int is2(int *arr, int max, int index)
{
    if (index < 0)
        return false;
    if (arr[index] == key)
        return true;
    return is2(arr, max, index-1);
}
```

```
int is(int *arr, int N)
{
    return is2(arr, N-1, N-1);
}
```

```

    ↗ getMax
    ↪ int gm(int* arr, int N, int max)
    {
        if (arr[max] > arr[N-1])
            max = N-1;
        if (N-1 == 0)
            return arr[max];
        return gm(arr, N-1, max);
    }

```

Recursive Code GCD :- ↗ Greatest Common Divisor

```

    ↪ int gcd(int d, int n)
    {
        int rem = n % d;
        return gcd(rem, d);
    }
    ↪ if (rem == 0)
        return d;

```

Recursive Code Decimal to Binary

```

void dtb(int n)
{
    int rem = n % 2;
    dtb(n/2);
    cout << rem;
}
    
```

~~if (n < 2)~~
~~return;~~

Console:-
for 18
 10010

No need to do this
 excessive work
 hai.

Recursive Code for Binary Search:-

```

int bs (int *arr, int l, int u, int key)
{
    if (l > u)
        return -1;
    int mid = lower + (upper - lower) / 2;
    if (arr[mid] == key)
        return key;
    else if (arr[mid] < key)
        return bs(arr, mid + 1, u, key);
    else
        return bs(arr, l, mid - 1, key);
}
    
```

Date _____

→ EOF Manipulation
in makeup class

Day _____

~ (Lecture nos 12) ~

- PrintSquares function done in class.
- Str overloaded String extraction operator (>>) using the whitespace logic.
- Windows uses 2 bytes to store '\n' & '\r' whereas Linux (Mac) uses 1 byte.
- '\r' → carriage return.

↳ Island Question practice File:-

```
void explore(int**a, int i, int j)
```

↓
we'll use our own
array.

```
void explore(Array<Array<int>>, int i, int j)  
{
```

```
    a[i][j] = 2;
```

```

if (i-1 >= 0 && a[i-1][j] == 0)
    explore(a, i-1, j);
if (j-1 >= 0 && a[i][j-1] == 0)
    explore(a, i, j-1);
if (j+1 < a[0].getSize() && a[i][j+1] == 0)
    explore(a, i, j+1);
if (i+1 < a.getSize() && a[i+1][j] == 0)
    explore(a, i+1, j);
}

```

→ Above has same time and Space complexity as stack maze code.

↳ `bool explore(Array<Array<int>> a, int i, int j, int di, int dj).`

- base case:

```

if (i == di && j == dj)
    return true;

```

- and if previous paths have returned true we'll not do further recursion calls.

- We'll take `bool status = false;`

- At the end, return status.

↳ If recursive ($\overset{\text{Time}}{\uparrow} T \cdot \overset{\text{Space}}{\uparrow} S$) == iterative ($T \cdot S$), then we should go for the recursive code. Else go with one having better time & space complexity.

◦ `int explore(Array<Array<int>>a, int i, int j)`

- we'll take `int count = 0;` and count the total recursive calls being generated at end we return count.

↳ The above gives size of each island. (one at a time).

↳ Static variable usage in recursion is strongly prohibited.

↳ Extra parameters bhi use nhi krna.

↳ No extra functions until allowed.

Date _____

Day _____

→ Dynamic Programming.

↳ DP can do all these problems in Polynomial Time.

Date _____

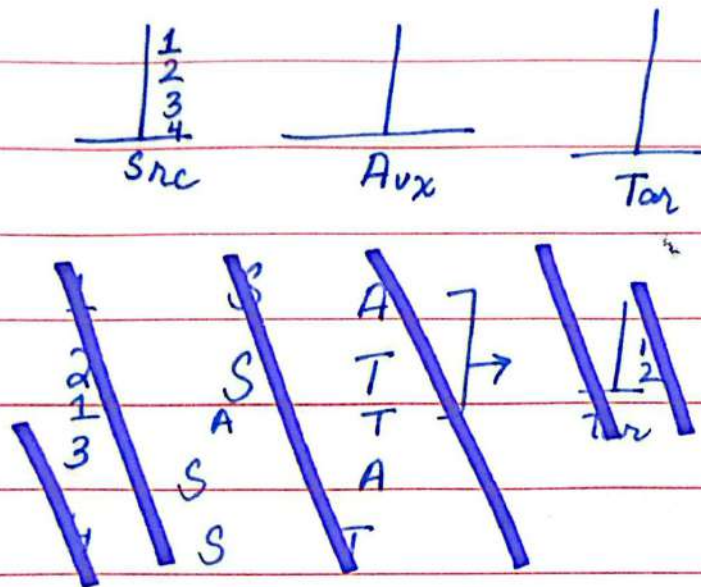
Day _____

~ (Lecture: no 13) ~

1) Self Referential Object:

2) Recursion $\xleftarrow{\text{toh}}$ Time/Space.

TOH:-



↳ Source & Target change according to moves.

* void toh(char Src, char Aux, char Targ, int N)

Date _____

∴ When disc == 1, move source to Target.

Day _____

Recursive Trace for TOH:-

$\swarrow$ $\text{toh}('S', 'A', 'T', 1)$
 $\text{toh}('S', 'A', 'T', 3) \rightarrow \text{toh}('S', 'T', 'A', 2)$
 $\swarrow$ $\text{toh}('A', 'S', 'T', 2)$
 $\swarrow$ $\text{toh}('T', 'S', 'A', 1)$
 --- and so on.

↳ If (N==1)

cout << src << "To" << Tar << N;

↳ void toh (char src, char Aux, char Tar, int N)

{

if (N==1)

cout << src << "To" << Tar << N;

toh (src, Tar, Aux, N-1)

↳ cout <<

toh (Aux, src, Tar, N-1);

}

Time / Space equation of Recursion:-

void f(int N)

{ if (N <= 0)

return;

Date _____

Day 1

$$\overset{N}{f(4)} - \overset{N}{f(3)} - \overset{N}{f(2)} - \overset{N}{f(1)} - \overset{N}{f(0)}$$

↳ Time Equation = $3N + 2$

int N (No. of Times)

faida ho.

Iteration

↳ dtb : $(T.S)_{recursive} = (T.S)_{iterative}$.

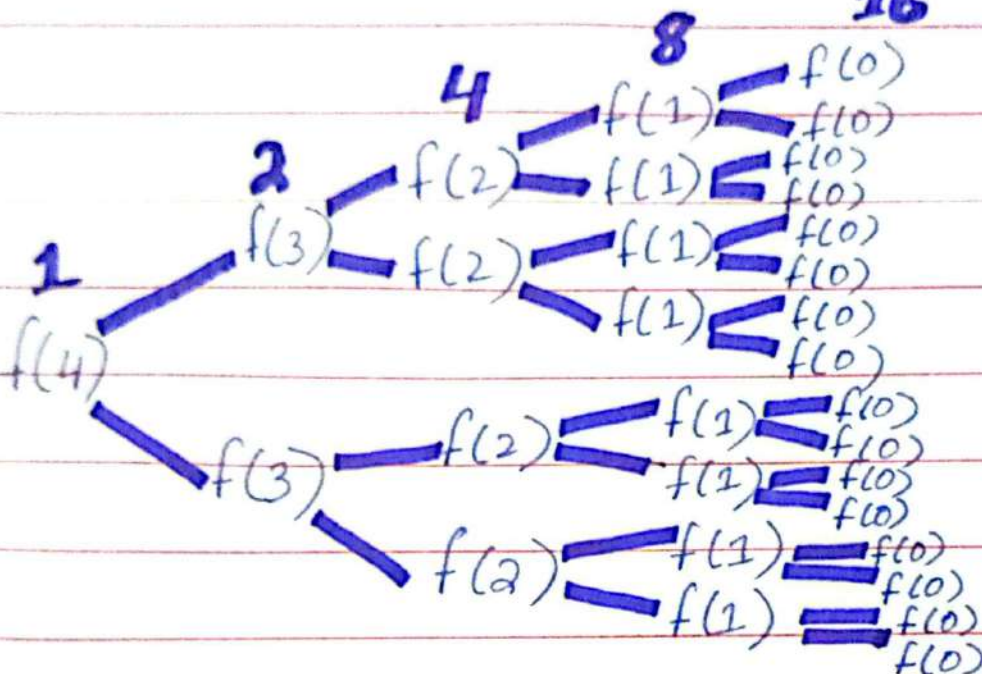
3

Recursive Trace:-

$$\sum_{n=0}^{\infty} \frac{r^{n-1}}{n-1}, \quad \left(\frac{2^{N+1}}{-1} \right)$$

16

↳ recursive calls



$$\hookrightarrow \frac{3 \cdot 2^{N-1}}{2 \cdot 2^N} + \hookrightarrow O(2^N) \rightarrow \text{exponential function.}$$

Time Equation.

$$\hookrightarrow 4(N+1)$$

$$\hookrightarrow O(N) \rightarrow \text{Space Equation.}$$

↳ Time Equation of exponential function fibonacci is 2^N and 2^{N+1} (Upper Bound).

$$\hookrightarrow \text{Maze}(R)(T \circ \beta) = \text{Maze}(\text{I})(T \circ \beta)$$

↳ Mostly Compilers convert tail recursion into iterative code while compilation.

Self Referential Object:-

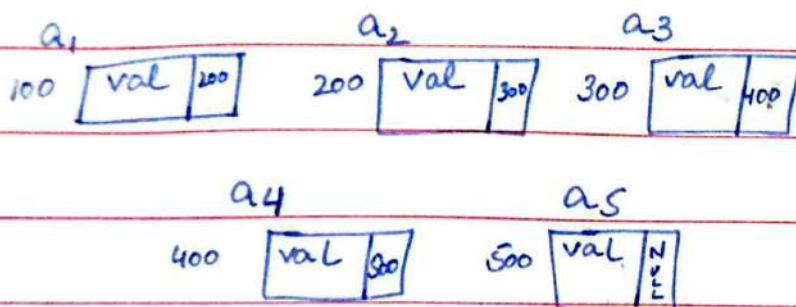
```
struct X
{
```

```
    int val;
```

```
    X * ptr;
```

```
}
```

↳ X a₁, a₂, a₃, a₄, a₅; → in main



∴ Diagram

Note:-

① a₁;
↓
X

② * &a₁;
↓
X

③ &a₁;
↓
X *

④ &*a₁;
↓
Syntax Error

Date _____

Day _____

↳ as.ptr → ptr;



To access nameless
object.

Date _____

Note :- $\rightarrow$

L $\rightarrow$ R

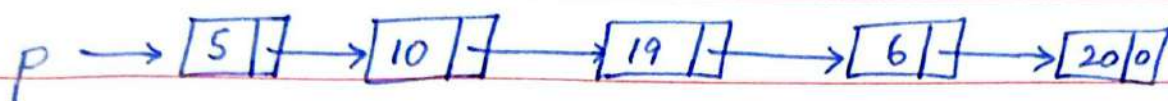
Precedence

Day _____

◦ (Lecture : no : 14) ◦

Self Referential Object :-

∴ Alternate of Array. (Linked Structure)



◻ To print all values :-

```
void print (X * p)
```

// To display.

```
{ while (p)
```

```
{
```

```
    cout << p → info << " : ";
```

```
    p = p → ptr;
```

```
}
```

```
}
```

→ This structure is complement of Array.

→ No shifting in this structure.

→ Array is a fixed size structure.

→ Linked List is a constant time growable / Shrinkable structure.

```
* bool search (X * p, int key)
{
```

```
    if (!p)
        return false;
```

∞ Linear Search
on Linked
Structure.

```
    X * q = p;
    while (q)
```

```
    {
        if (q->val == key)
            return true;
        q = q->ptr;
    }
```

```
    return false;
}
```

```
* void insertAtTail (X * p, int val)
{
```

~~return false;~~

```
    X * q = p;
    while (q->ptr)
        q = q->ptr;
```

```
    q->ptr = new X;
```

```
    q->ptr->ptr = nullptr;
    q->ptr->val = val;
```

```
}
```

∞ Insertion at
Tail.

```
if (!p)
```

```
{
```

```
    p = new X;
```

```
    p->ptr = nullptr;
```

```
    p->val = val;
```

```
}
```

Linked Structure can also be made using Array.

Date _____

Day _____

→ We'll use Array or Linked Structure according to our requirements and benefits.

- 1) List → (Grocery List, Merit List) → queue, stack is also a variation of this.
- 2) Array List → concept of list implemented using array structure.
- 3) Linked List → concept of list implemented using linked structure.

Linked List:-

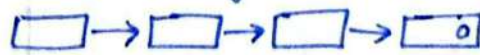
→ Concept of List implemented using linked structure.

→ Memory Consumption of Linked List is more than Array. (We also consume pointers with each Node).

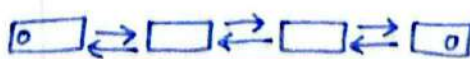
Types:-

Diagrams:-

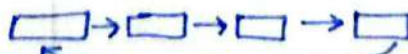
1) LSL



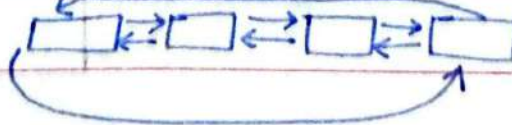
2) LDL

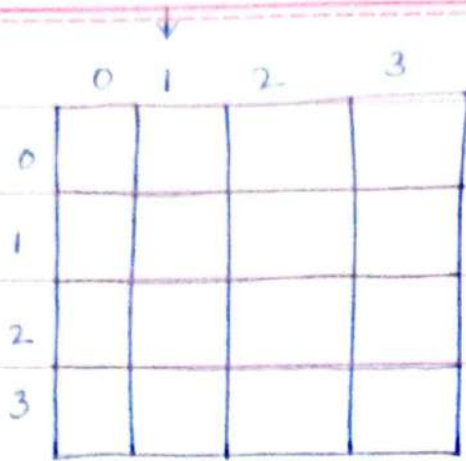


3) CSL

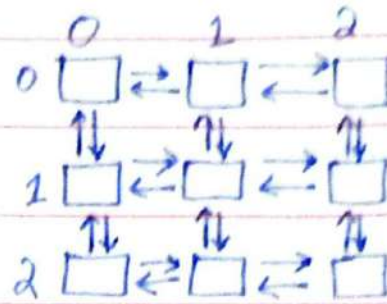


4) CDLL





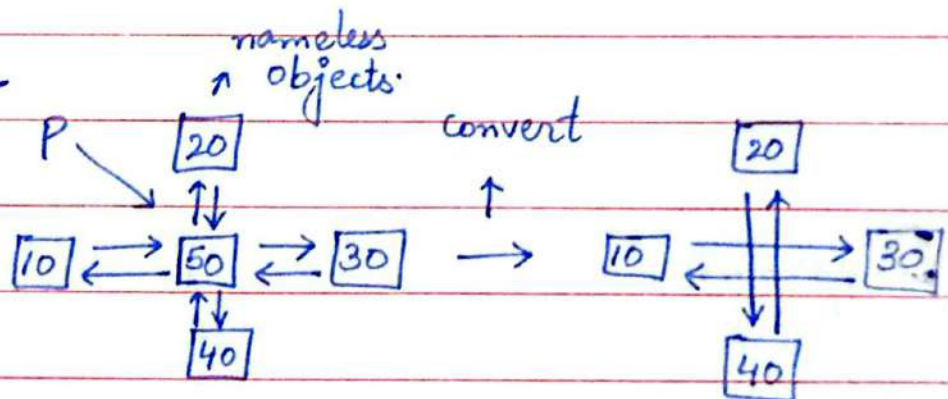
:: Diagram 1 ::



:: Diagram 2 ::

→ If we want to delete ~~column~~ column 1 we've to do much shifting in array structure but not in linked structure. In Linked List it'll be a constant time $O(1)$ operation.

Example :-



• Convert without using extra pointers.

Solution :-

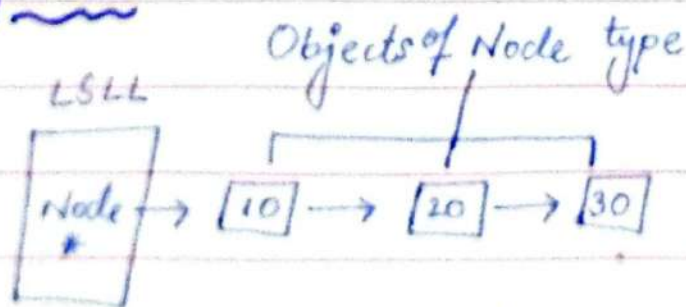
1. $p \rightarrow \text{left} \uparrow = p \rightarrow \text{right}$ → right
2. $p \rightarrow \text{right} \uparrow = p \rightarrow \text{right} \rightarrow \text{left}$ → left
3. $p \rightarrow \text{top} \uparrow = p \rightarrow \text{bottom}$ → bottom
4. $p \rightarrow \text{bottom} \uparrow = p \rightarrow \text{bottom}$ → top
5. delete p ;

Node:-

```

template <typename T>
struct Node
{
    T info;
    Node<T> * next;
    Node(): next(nullptr)
    {}
    Node(T val): info(val), next(nullptr)
    {}
};

```

Diagram:-LSLL:-

```

template <typename T>
class LSLL

```

{

Node < T > * head;

public:

LSLL(): head(nullptr)

{}

void insertAtHead(T val);

void insertAtTail(T val);

void insertAfter(T key, T val);

void insertBefore(T key, T val);

void removeAtHead();

void removeAtTail();

void removeAfter(T key);

void removeBefore(T key);

void remove(T key);

~ LSLL();

// Copy Constructor

// Copy Assignment

∴ First write code
for generic cases
then deal with
exceptional
cases.

→ // isEmpty function.

};

* void insertAtHead(T val)

{

if(!head)

{

→ it'll also work
without this
condition.

head = new Node<T>(val);

}

~~Node<T> *p = head;~~

~~{~~

Node<T> *p = new Node<T>(val);

p → next = head;

head = p;

}

* void ^{insert} ~~Before~~Before(T key, T val)

{

if (!head)

return;

Node<T> *p = head;

while (p → next != nullptr && p → next → info != key)

{

p = p → next;

}

if (p → next == nullptr)

return;

Node<T> *x = new Node<T>(val);

x → next = p → next;

p → next = x;

}

if (head → info == key)

{

InsertAtHead(key);

return;

}

```
* void removeAtHead()
{
```

```
    if(!head) //use isEmpty() instead of this.
        return;
```

```
    Node<T>* p = head;
```

```
    head = head->next;
```

```
    delete p;
```

```
}
```

```
* void removeBefore(T key)
{
```

```
    if(!head)
```

```
        return;
```

```
    else if (head->next->val == key)
```

```
    {
```

```
        head->next && removeAtHead();
```

```
        return;
```

```
    }
```

```
    Node<T>* prev = head;
```

```
    Node<T>* p = head->next->next;
```

```
    while (p != nullptr && p->info != key)
    {
```

```
prev = prev->next;  
p = p->next;  
}
```

```
delete prev->next;  
prev->next = p;  
}
```

➤ Practice:-

- 1) Sort List using Selection Sort. (Change info)
- 2) Sort List using selection Sort. (Change Nodes) (Link part will be changed)
- 3) Middle of Linked List in only one traversal.
- 4) Tell if a linked List is circular or not.
- 5) Tell point of intersection of two linked list.

Date _____

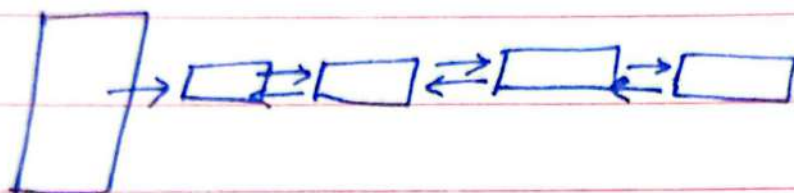
 Do:-
 1) Vector.
 2) Stack.
 3) Queue.

Day _____

◦ (Lecture: no: 15) ◦

* list of STL == LSL (Linear Single Linked List)

◦ LDLL:-



↳ struct DNode

```
{
    T info;
    DNode <T> * next;
    DNode <T> * prev;
}
```

* class LDLL

```
{
    DNode <T> * head;
public:
```

```
    LDLL() { head = nullptr; }
```

```
    // all functions just like LSL
```

```
};
```

```

o void insertAtHead()
{

```

```

    DNode <T> * x = new DNode<T>(val);

```

```

if if (!head)

```

```

    { head = x;

```

```

      return;

```

```

    }

```

```

    x->next = head;

```

```

    head->prev = x;

```

```

    head = x;

```

```

}

```

```

o void removeBefore(key key)
{

```

```

    if (!head ||
        head->next
        == nullptr)
        return;

```

```

    DNode <T> * p = head;

```

```

    while (p != nullptr && p->info != key)

```

```

        p = p->next;

```

```

    if (!p || p->next == nullptr)
        return;

```

```

    if (p == head->next)
    {
        removeAtHead();
        return;
    }

```

```

    DNode <T> * x = p->prev;

```

```

    x->next->next = p;

```

```

    p->prev = x->prev;

```

delete x;

}

* COLL:-

```
void print()
{
```

```
    DNode<T>* p = head;
```

```
    do
    {
```

```
        cout << p->info << ' : ';
```

```
        p = p->next;
```

```
    }
```

```
    while(p != head);
```

```
}
```

```
* void insertAtHead()
{
```

```
    DNode<T>* p = new DNode<T>(val);
```

```
    p->next = head; → if (!head)
```

```
    { head = p;
```

```
    p->prev = head->prev;
```

```
    head->prev =
```

```
    head->prev = p;
```

```
    head->next = head;
```

```
    return;
```

```
    p->prev->next = p;
```

```
}
```

```
    head = p;
```

```
}
```

* void removeAtTail()

{

→ if (head == head → prev)
{ delete head;

head = nullptr;
return;

DNode<T> * x = head → prev;

head → prev → ~~prev~~ prev → next = head;

head → prev = x → prev;

delete x;

if (x == head)

{ delete x;

head = nullptr;

return;

}

Date _____

1) list $\rightarrow$ DLL
2) forward-list $\rightarrow$ LSL

Day _____

(Lecture no: 16)

we'll use LSL in place of this.

* `vector<int> v;`

object of Iterator class.

1) `vector<int>::Iterator i = v.begin();`

`while (i != v.end())`

{

*
`cout << *i;`

`i++;`

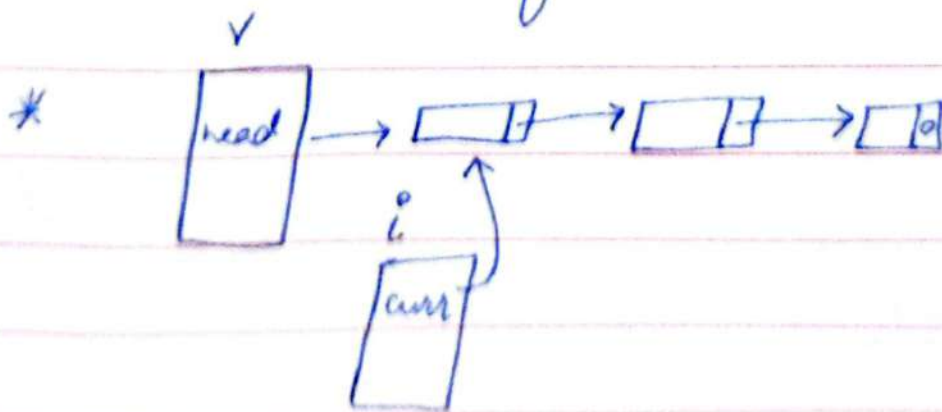
}

Nested class.
Iterator can also be made outside class but STL used nested class concept.

2) `for (auto i : v)`

`cout << i << " ";`

$\hookrightarrow$ Iterator is used because we can't define every functionality in the class itself.



◦ Nested Classes:-

```
class A
{
    int a1;
    class B
    {
        int b1;
        public:
        int b2;
    };
    void f()
    {
        // statements
    }
};
```

◦ Name Space:-

namespace

~~MA~~ ABC

{

class Student { };

};

→ Now used 'using namespace std' for Student a; rather than ABC::Student a;

- * C# also has namespace.
- * In Java it is package.
- * forward-list has forward Iterator.
- * vector has bidirectional Iterator
- * We made Iterators Nested so there shall not be Iterators of two classes having same name.

↳ Iterator (LSLL<int>&l)

{

cur = l.head;

}

→ it'll cause error because l.head is private so we'll make Iterator friend of LSLL.

↳ T& operator*()

{ if (cur)

return cur->info;

exit(0);

}

→ void operator++() ^{→ (pre)} :: ++i;
{
 if (cur)
 cur = cur → next;
 exit(0);
}

→ void operator--() ^{→ (pre)} :: --i;
{
 ↑
 additional
 ↑
 :: for LDLL
 if (cur)
 cur = cur → prev;
 exit(0);
}

→ There'll be no copy constructor and destructor in Iterator class. (Shallow Copy is being done)

→ ++ operator should (be) × be overloaded in Lists because they aren't consecutive.

◦ Concept of friend is only in **C++**.

* **!=** also overloaded in **Iterator**.

* In case of returning nested classes in templates. We also have to **write** typename before the prototype.

↳ 1) **begin()**/**end()**. (forward)

2) **rbegin()**/**rend()**. (reverse)

3) **cbegin()**/**cend()**. (constant)

⇒ Linked List Using Array:-

◦ Insertion at middle of Linked List is $O(1)$ and in array its $O(n)$. we aren't adding the searching cost if we add searching cost then both are $O(n)$.

- Insertion at tail of Linked List is $O(1)$ if we maintain a pointer at end else it'll be $O(n)$.

(int) info ← 0 1 → link(int)

0		
1		
2		
3		
4		

↳ -1 == nullptr in this structure.

↗ class
LSLL
{

Node<T> data[8];

int head;

public:

LSLL () { head = -1; }

// rest functions as it is.

};

```
void insertAtHead(T val)
{
```

```
    int p = getNewNode();
    data[p].next = head;
    data[p].info = val;
    head = p;
}
```

```
void insertAtTail(T val)
```

```
{
    if (head == -1) {
        insertAtHead(val);
        return;
    }
    int p = head;
```

```
    while (data[p].next != -1)
        p = data[p].next;
```

```
    int x = getNewNode();
```

```
    data[p].next = x;
```

```
    data[x].next = -1;
```

```
    data[x].info = val;
}
```

↳ While deleting place -2 in the linked part.

↳ new == $O(n)$. (How can we do it in $O(1)$?)
↳ by us.

↳ delete == $O(1)$.

- A Graph with no cycle is a tree.

◦ (Lecture no 17) ◦

- ↳ we make a new list named avail containing all the available nodes. (in this the memory cost is $O(1)$ where in ~~the~~

~~Stack~~ Stack solution
the memory cost is $O(n)$
(The O of both the solutions
in T.C is $O(1)$).

- ↳ In iterators of CSLL & COLL take two pointers one is pointer and the other will be constant pointing to the head and change implementation of ++ operator.

- ↳ Nested class has the right to change private part of host class but it isn't true

Date _____

↳ Robb, Book, Author, Edges/Vertices → Excel file.
Day _____

Vice versa.

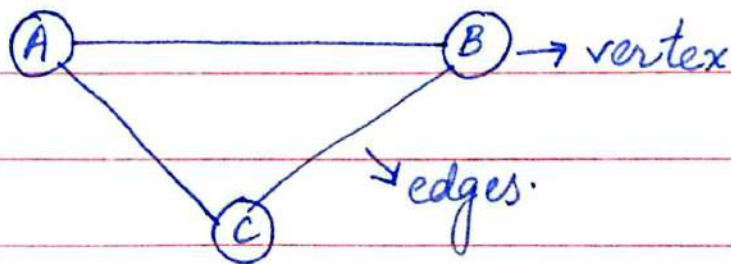
* Iterator (LSLL <int> * l)

{

cur = l → head;

}

Graph:-



↳ Set of vertices connected through edges.

↳ No. of edges is the length of the path in graph.

↳ In cycle, the starting and ending node of a path is same.

↳ In a path we can't use the same edge twice.

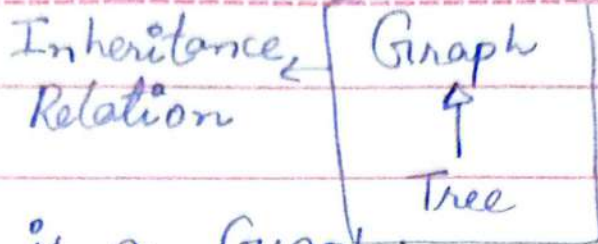
is Sibling & brother same?

Google it. → Graph Algo.
Perfect Matching Algorithm.

Date _____

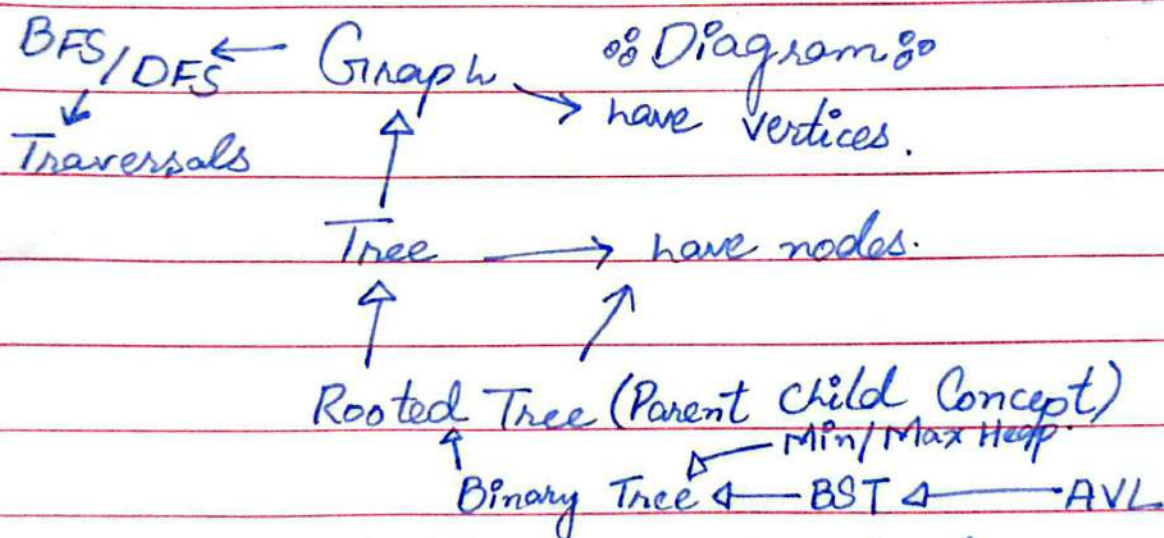
Day _____

Free:-



- * Every tree is a Graph.
- * A Graph with no cycle is a tree.
- * A Tree is never disconnected.

↳ In a tree, if there are 5 vertices then there'll be 4 edges.



↳ In Rooted Trees, shape is critical.

↳ Leaf Node : Having no child.

↳ Root Node can also be leaf Node.

↳ Levels of Rooted Tree.

↳ Degree of Node. (No. of child of a Node)

↳ Height of tree. (No. of Levels of Tree)
↳ The longest path from root to leaf.

* Level 0 or Level 1?

Date _____

Day _____

↳ Depth of a Node. (Level no.)

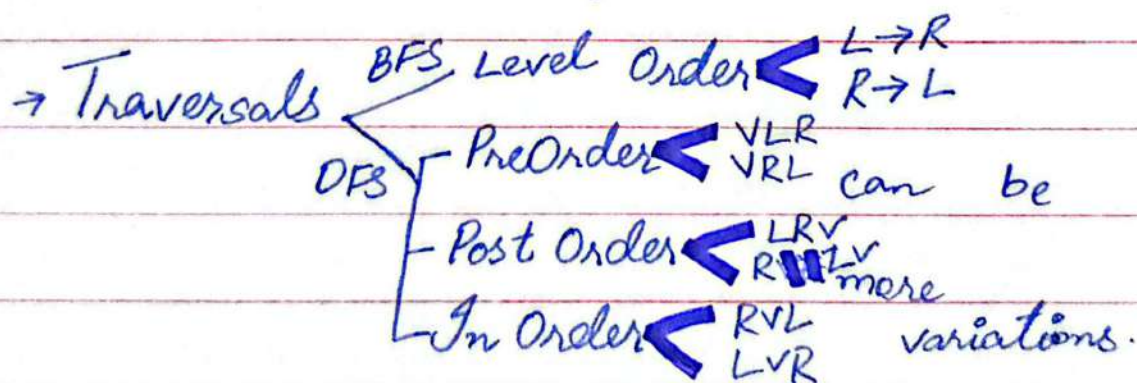
Binary Tree:-

↳ Every Binary Tree is a Rooted Tree.

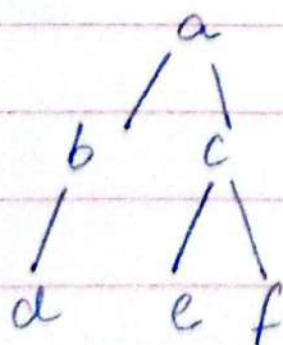
↳ A Node can have maximum degree 2.

↳ Distinction of Left & Right Child.

↳ Traversals of Tree.



↳ Level Order Traversal:-



Order

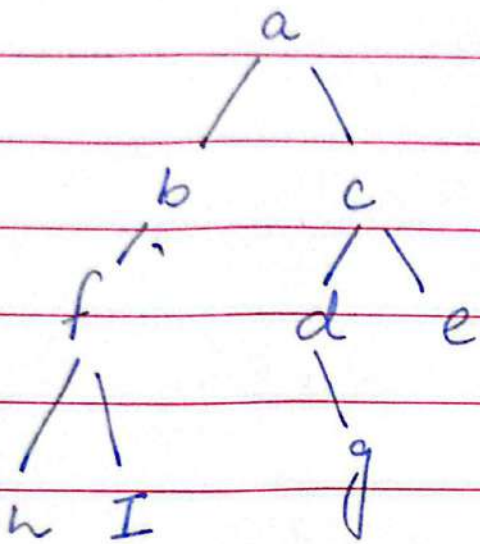
Level $\uparrow$ = a b c d e f

In ~~Pre~~ Order = d b a e c f

Pre Order = a b d c e f

Post Order = d b e f c a

◦ Example:-



∴ Do Practice of Traversals.

↳ $2^h - 1$ → maximum no. of nodes

in binary tree of height h .

↳ h → minimum number of nodes:

in binary tree of height h .

↳ n → max height having n nodes.

↳ $\downarrow$ → min height having n nodes.

$$\lceil \log_2(N+1) \rceil$$

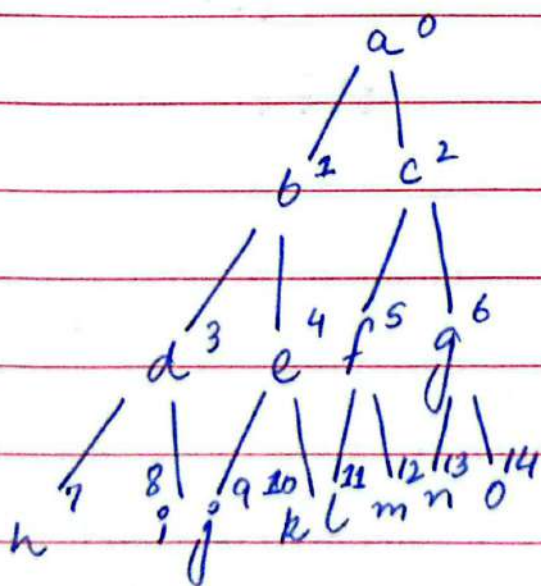
(Lecture no: 18)

- 1) Full Binary Tree. $\rightarrow$ if height is h then $\text{nodes} = 2^h - 1$
- 2) Complete Binary Tree.

$\swarrow$ The last level
 nodes should
 be left to right
 consecutive and
 second last level
 should be Full
 Binary Tree.

Array Binary Tree:-

$\therefore$ Path is no. of
 nodes and
 height
 start from
 1 in
 our
 implementation



- $\text{leftChild}(i) = i + i + 1 = 2i + 1$
- $\text{rightChild}(i) = i + i + 2 = 2i + 2$

Date _____

Day _____

* left childs are at odd indexes and right child are at root node.

```
class BinaryTree
{
```

```
    T* data;
```

```
    int maxHeight;
```

```
public:
```

```
    int arrayCapacity;
```

```
    bool* NodeStatus;
```

```
    BinaryTree(int h)
    {
```

```
        maxHeight = h; arrayCapacity = pow(2, h) - 1;
```

```
        data = new T[pow(2, h) - 1];
```

```
    }
```

do same for Node Status. arrayCapacity here. (initialized by false)

```
};
```

* To get Descendents $\frac{P-1}{2} = li$  same for both.

```
void VLR(int i) O(n) → Time  O(h) → Space
```

```
{    if(NodeStatus[i] == false)
    {
        return;
```

```
        cout << data[i];
```

```
        VLR(2*i+1); } if(formula < arrayCapacity)
```

```
        VLR(2*i+2);
```

```
    }
```

&& NodeStatus [Formula] = true;

↳ for Level order we can do simple array display but its O will be much high (height is 20 but only 2 nodes are there so it'll still visit every node)

⇒ Level Order Traversal:-

◦ we'll take a queue and push root into it and push its children then pop it and display.

We'll do this operation until queue isn't empty. (TC = $O(n)$, SC

= $O(2^h)$ ∴ precise space = $2^{h-1} \mid N/2$

SC = $O(n)$

roughly
this number of
Nodes in last
level.

↓ ~~not~~ ~~not~~ ~~not~~

```
↳ int search (int i, T key)
{
    int index;  $\rightarrow -1$ 
    if (data[i] == key)
        index = i; index = i;
    if (index == -1 && // bound checking for left)
        index = i;  $\rightarrow$  i = call at left.
    if (index == -1 && // bound checking for
        right)
        index = i;  $\rightarrow$  i = call at right.
    return index;
}
```

```
↳ void setLeftChild (T par, T child)
{
    int index = search(0, par);
    if (ind == -1)
        return;
    if ( $2 * \text{index} + 1 \geq \text{arrayCapacity}$ )
        return;
    data[ $2 * i + 1$ ] = child;
    nodeStatus[ $2 * i + 1$ ] = true;
}
```

↳ void remove(T node)

{

int index = search(0, node);

// nodeStatus of node and its
descendents will be false. (make
a private function for it.

}

↳ int getHeight(int i)

{ if($i \geq \text{arrayCapacity}$) → return 0;

if ($(2*i+1 \geq \text{arrayCapacity} \ \&\& \ 2*i+2 \geq \text{arrayCapacity}) \ || \ (\text{nodeStatus}[2*i+1] == \text{false} \ \&\& \ \text{nodeStatus}[2*i+2] == \text{false})$

return 1;

int LHeight = ~~0~~, RHeight = ~~0~~;

if ($2*i+1 < \text{arrayCapacity} \ \&\& \ \text{nodeStatus}[2*i+1] == \text{true}$)

LHeight = getHeight($2*i+1$);

if ($2*i+2 < \text{arrayCapacity} \ \&\& \ \text{nodeStatus}[2*i+2] == \text{true}$)

RHeight = getHeight($2*i+2$);

Date _____

Day _____

```
if (LHeight == -1 && RHeight == -1)
```

```
return 1;
```

```
return 1 + max(LHeight, RHeight);  
}
```

⇒ Linked Binary Tree:-

↳ Struct BTreeNode

```
{
```

```
    T info;
```

```
    BTreeNode<T> * left;
```

```
    BTreeNode<T> * right;
```

```
};
```

↳ class LinkedBinaryTree{

```
    BTreeNode<T> * root;
```

```
public:    Binary
```

```
    Linked↑Tree()
```

```
{
```

```
    root = nullptr;
```

```
}
```

```
};
```

Day

~ (Lecture no: 19) ~

➤ Binary Search Tree :-

and Searching in $O(\log n)$.

• (Lecture 8 nos 20)

↳ AVL stands for Adelson Velski
Lendis.

↳ BST $\rightarrow O(n)$ $\rightarrow$ Not always $O(\log N)$

N $\log_2 N$

↳ AVL $\rightarrow O(h) \rightarrow$ always $O(\log N)$

$$\log_2 N$$

↳ Additional thing in AVL is
Balance Factor. (it is in range ± 1)

◦ $\text{BalanceFactor} = \text{LeftSubTree}(h) - \text{RightSubTree}(h)$

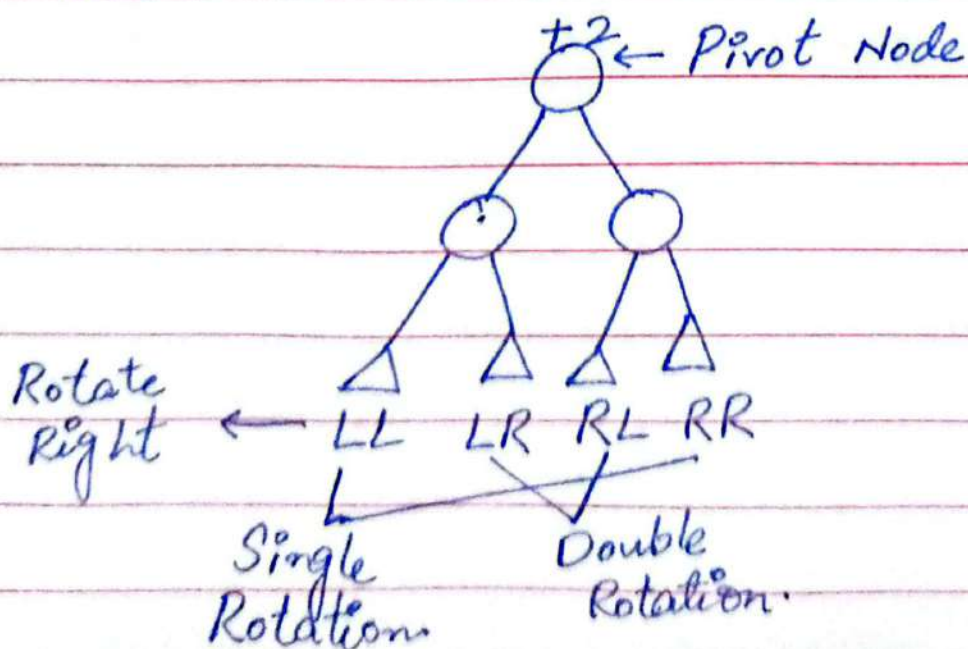
◦ BST and AVL are both search Trees.

◦ We do restructuring in AVL.

◦ Due to insert ~~insert~~ and delete the maximum height that can go up and down is by 1.

◦ Tree of height 1 and 2 are always balanced.

◦ Pivot Node is the Node where balance factor is out.



↳ In AVL our BTreeNode have another data member named as height.

↳ Use Stack to get the parent Node.

→ LL :-

LP → right = P;

p → left = RLP;

If (p == root)

root = LP;

else if (pp → left == P)

pp → left = LP;

else

pp → right = LP;

↳ Also do ~~HH~~ recalculation of height after rotation.

↳ Dogleg → LR & ~~HH~~ RL

↳ No Duplicate values are allowed in AVL.

Date _____

Day _____

		Roll no	Name
0	1	15	—
1	15	30	—
2	20	70	—
3	30	1	—
4	36	36	—
5	70	20	—

↓
Add whole (n^2) data, ~~delete~~ delete(n), insert(n)

↳ if we use tree, Add whole data $(n \log n)$,
~~delete~~ delete $(\log n)$, insert $(\log n)$.

↳ delete is heavy operation in DB
if implemented by Tree.

Lecture no: 20

1

AVL:-

RR:-

$RP \rightarrow \text{left} = P;$

$P \rightarrow \text{right} = LRP;$

// parent settings same as LL

LR:-

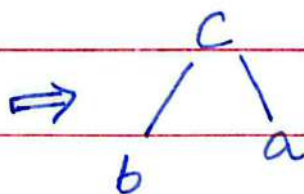
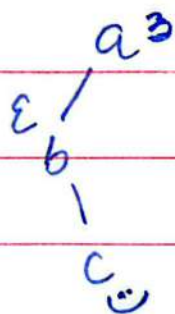
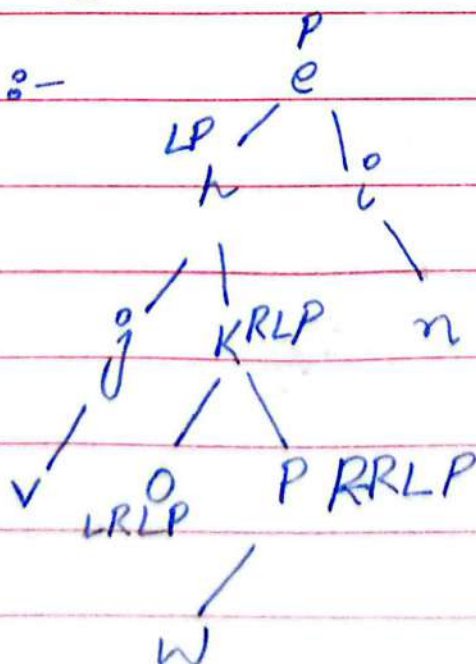


Diagram:-



Date _____

Day _____

left

↳ $RLP \rightarrow \text{left} = LP;$

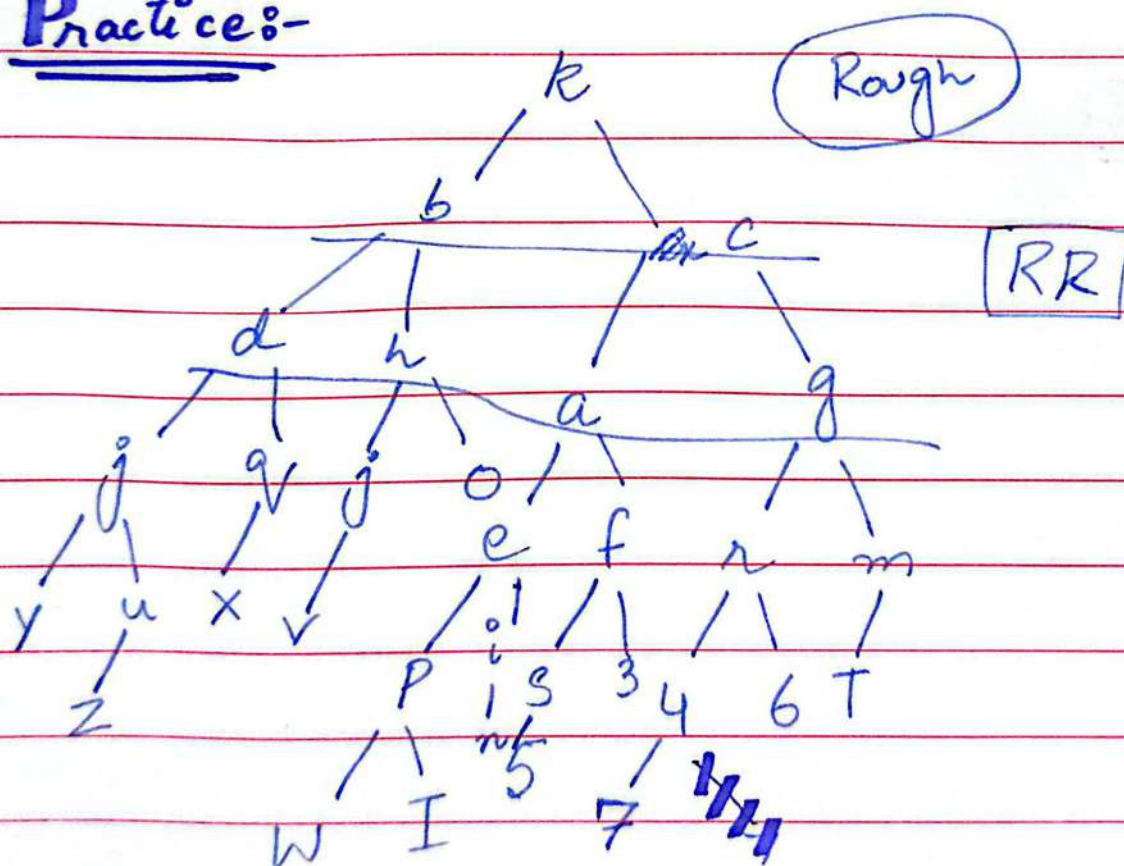
$RLP \rightarrow \text{right} = P;$

$LP \rightarrow \text{right} = LRLP;$

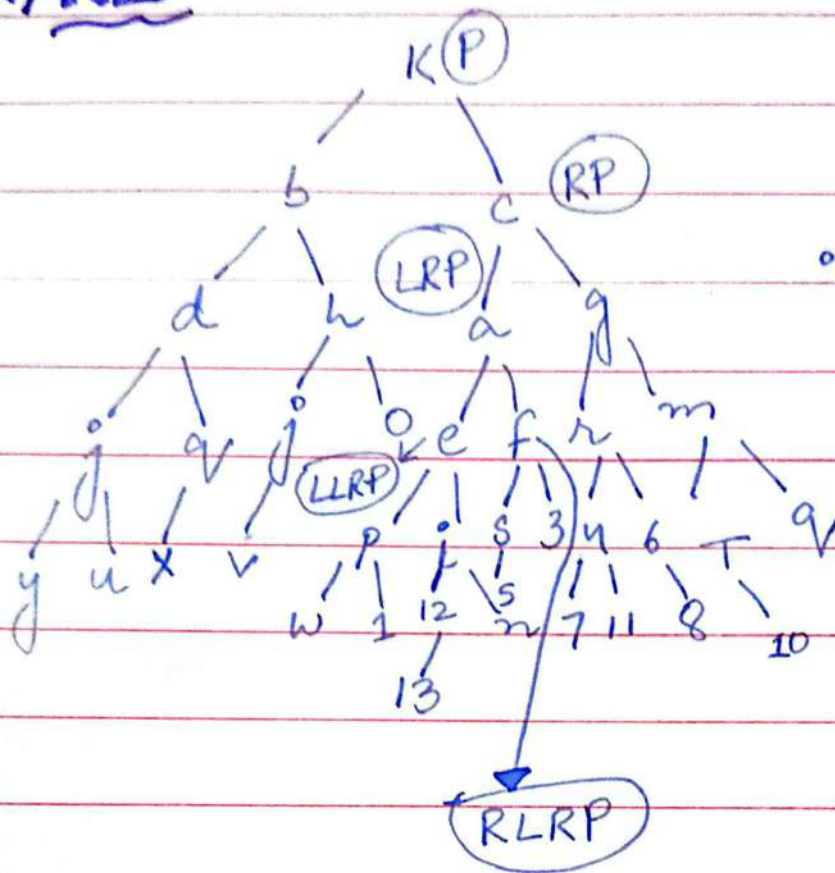
$P \rightarrow \text{left} = RRLP;$

// parent settings as ~~done~~ done previously.

Practice:-



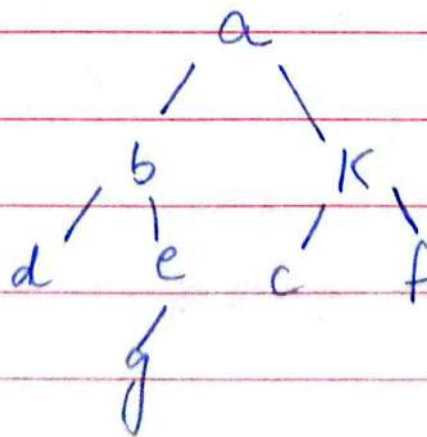
⇒ RL :-



∴ Mirror of LR

PRACTICE :-

①

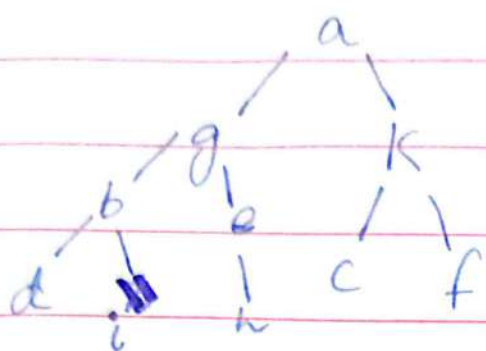


(RL)

Date _____

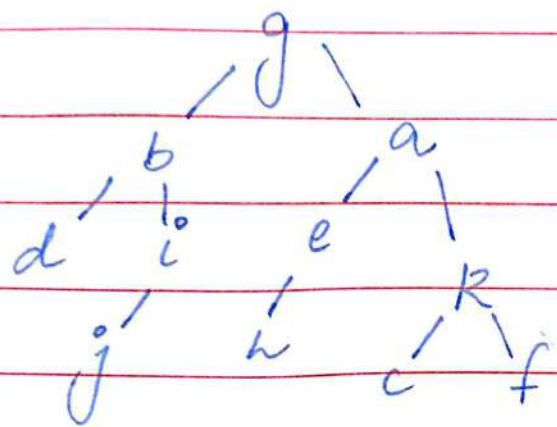
Day _____

(2)



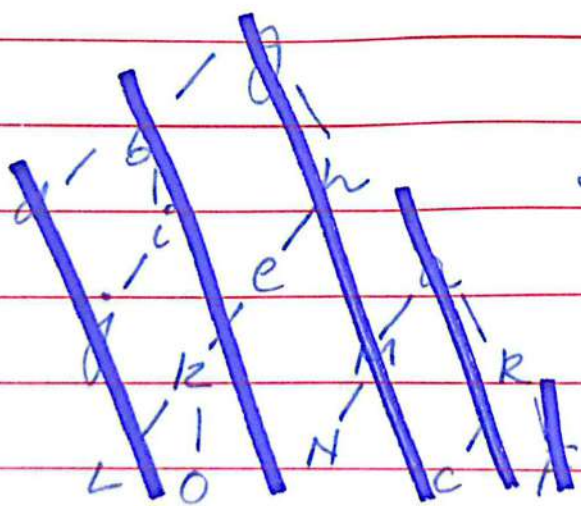
RL

(3)

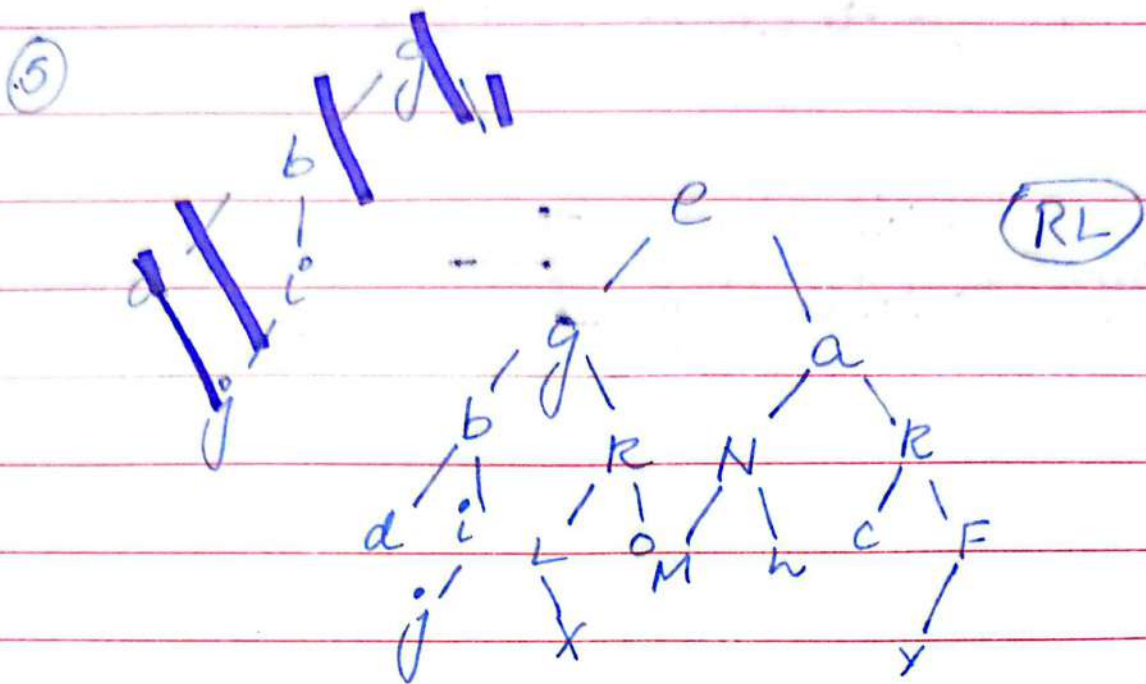


LL

(4)



∴ Simple LR Rotation.



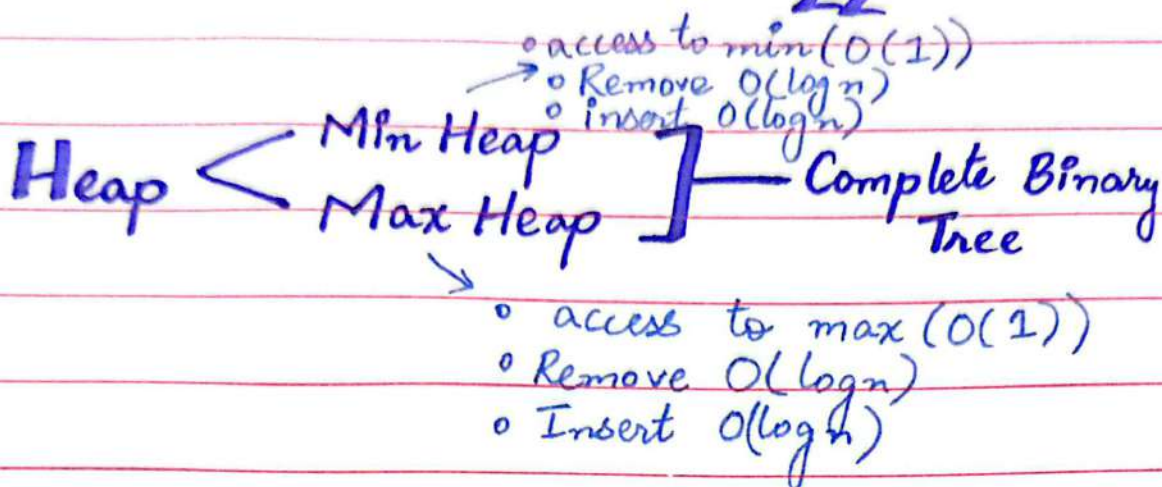
Deletion in AVL :-

↳ Check ancestor path of the physically deleted Node to make it AVL again.

Q. Write Test Case in which deleting a Node triggers 3 rotations in AVL?

↓
• past year's question.

(Lecture no: 22)



↳ minStack will do all three in $O(1)$ but it'll remove minimum in $O(n)$

MinHeap:-

- ↳ The node value is smaller than or equal to its subtrees.
- ↳ (Greater than in max heap).
- ↳ After insertion keep swapping until minheap/maxheap property isn't violated.

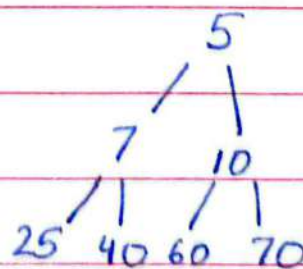
ADT:-

MinHeap

{

T * data;

int noe = 0;



◦◦ Min Heap.

int N; $\rightarrow$ Size of the Array

public:

MinHeap(int s)
{

N=s;

noe = 0;

data = new T[N];

}

void insert(T val);

T*(void)*removeMin();

T getMin();

bool isFull();

bool isEmpty();

int getNoe();

int getSize(); $\rightarrow$ Array

};

$\rightarrow$ Shift up / Heap up

oo Insertion Loop:-

$\rightarrow$ cur $\neq$ 0 &&

while (data[cur] < data[(cur-1)/2]) {

{

Swap(data[cur], data[(cur-1)/2]);

cur = (cur-1)/2;

}

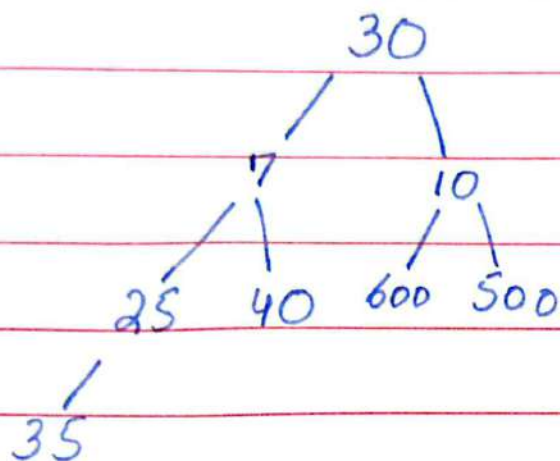
Date _____

Day _____

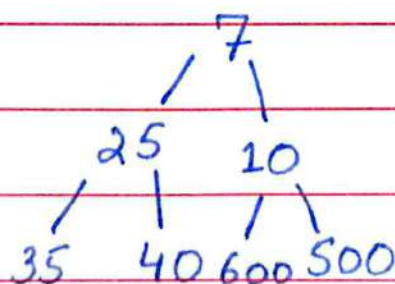
↳ In deletion we're doing shift down / heap down. (we'll go till noe)

(edge)

↳ Both Insertion & removal is called heapify.



After Remove Min
Operation.



∴ Deleted
30.

⇒ Remove Min() :-

↳ int getMin(int i)
{

if ($2 * i + 1 \geq \text{noe}$)

```
    return -1;  
    if (2*i+2 >= noe)  
        return 2*i+1;  
    return data[2*i+1] < data[2*i+2] ?  
        2*i+1 : 2*i+2;  
}
```

↳ T removeMin()

```
{  
    T min = data[0];  
    data[0] = data[noe-1];  
    noe--;  
    int cur = 0;  
    int minChild = getMinChild(cur);  
    while (minChild != -1 && data[cur] >  
        data[minChild])  
    {  
        swap ( , );  
        cur = minChild;  
        minChild = getMinChild(cur);  
    }  
}
```

}

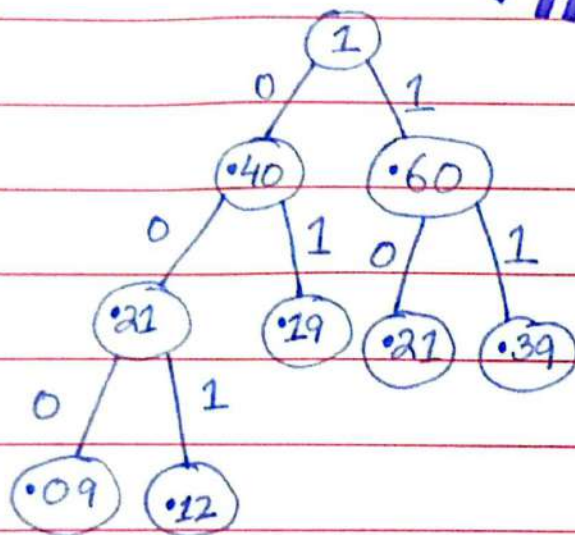
↳ Have to search min/max Heap
in STL.

➤ Data Compression

↳ Huffman Tree assigns unique codes in data compression.

↳ MinHeap < MaxHeap → This is done in

• **Huffman Tree Algorithm**



- I'll give unique codes.
- OS deals with primitive interrupts as soon as possible leaving every other operation.
- We can also implement priority queue using 2 or more queues.
- No FCFS in Max/Min Heap but it is $(\text{queue})^*$ guaranteed in

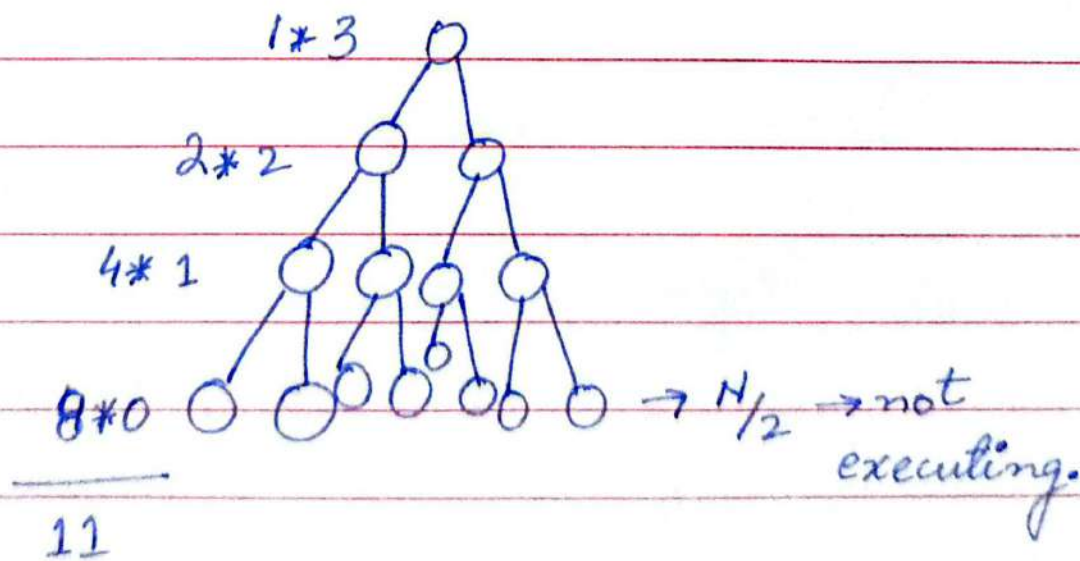
priority queue using two or more queues.

↳ if we want to make priority queue then we'll take another data ~~members~~ ~~timestamp~~ ~~(int)~~ and stamps (int *).

↳ Build Heap :- $\rightarrow O(1)$ Memory.

Convert array to a state that when it is seen in tree then it's Min/Max Heap.

↳ $T.C = O(n)$



$$\rightarrow 0 \cdot 2^{h-1} + 1 \cdot 2^{h-2} + 2 \cdot 2^{h-3} \dots + (n-1) \cdot 2^{h-n}$$

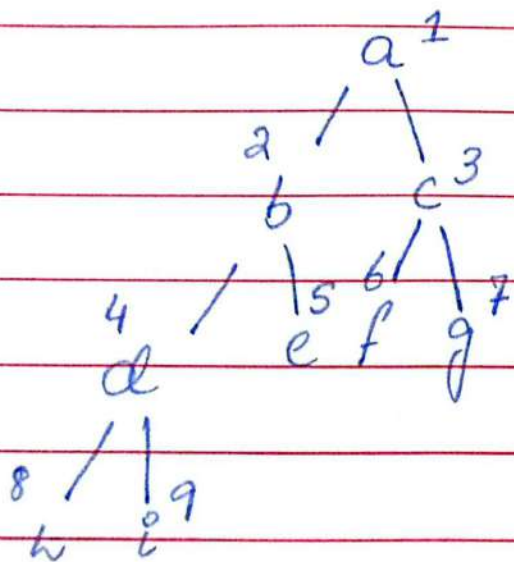
Date _____

Day _____

↳ Don't call function on ~~leaf~~ leaf nodes.

↳ Dijkstra's Algorithm's Complexity can't be achieved without Heap.

Linked Representation:-



∴ Just Divide by 2.

5 = 101 → Then we can simply go to the parent of node to be inserted.
6 = 110
9 = 1001
8 = 100

Lecture : no: 23

◦ SortedArrayToBST → must do on registers.

Graph :-

- ↳ Directed / Undirected.
- ↳ Connected / disconnected.
- ↳ Cyclic / Acyclic.

Already
Discussed
these terms.

↳ Adjacency Matrix / Adjacency list.

↳ $v(v-1) \rightarrow$ No. of edges in a graph of v vertices.

↳ In ~~Undirected~~ Directed graph, there are
 $\frac{v(v-1)}{2} \rightarrow$ No. of edges.

↳ Undirected graph is (conf)* confirm symmetric.

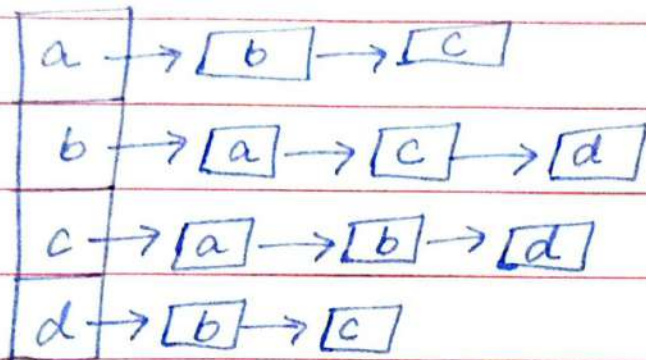
↳ ◦ Vertices = V .

◦ Edges = E .

↳ Memory consumption of adjacency Matrix is V^2 .

↳ It can be using extra space in adjacency matrix.

↳ In adjacency list, only that much memory will be used that is required.



↳ In coding, the vertices will be numbering.

↳ $O(1)$ cost of searching in Matrix.

↳ $O(n)$ cost of searching in List.

↳ $O(\frac{n}{2})$ cost to count in/out degree in Matrix.

↳ $O(n^2)$ cost to count in/out degree in ~~Matrix~~ List.

↳ DAG → directed **Acyclic** Graph.

```
class GraphAdjacencyMatrix
```

```
{    bool directed = true;
```

```
    Array<Array<int>> grp;
```

```
public:
```

```
    bool d = true;
```

```
    GraphAdjacencyMatrix(int v, ↑
```

```
    {    directed = d;
```

```
        grp.resize(v);
```

```
        for(int i=0; i<v; i++)
```

```
            grp.resize(v);
```

```
            for(int i=0; i<v; i++)
```

```
                for(int j=0; j<v; j++)
```

```
                    grp[i][j] = 0;
```

```
        }
```

```
    void addEdge(int u, int v, int wt=1)
```

```
    {    grp[u][v] = wt;
```

```
        if(!directed)
```

```
            grp[v][u] = wt;
```

```
    }
```

```
    void isEdge(int u, int v);
```

```
    void removeEdge(int u, int v);
```

```
    int indDegree(int u);
```

```
    int outDegree(int u);
```

```
void BFS();
```

```
};
```

```
struct Edge
```

```
{
```

```
int vertex;
```

```
int weight;
```

```
Edge * next;
```

```
};
```

→ // constructors.

```
class GraphAdjList
```

```
{
```

```
bool directed = true;
```

```
Array<Edge*> grp;
```

```
public:
```

```
GraphAdjList(int v, bool d)
```

```
{
```

```
    directed = d;
```

```
    grp.resize(v);
```

```
}
```

```
void addEdge(int u, int v, int wt)
```

```
{
```

```
    Edge * e = new Edge(v, wt);
```

```
    e->next = grp[u];
```

```
    grp[v] = e;
```

```
}
```

};

→ BFS in Graph:-

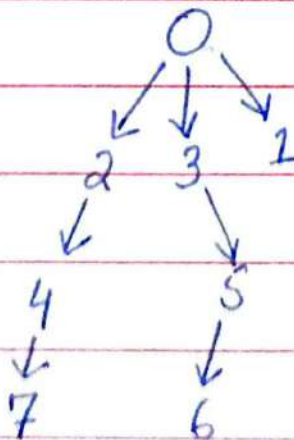
↳ Huge Margin of Performance of Adjacency List in case of Sparse Graph.

- We'll use a queue and use a boolean array for storing its visited status.

↳ Output 1:-

0 3 1 2 5 4 6 7.

↳ Output 2:-



T.C

void BFS (int sVertex) $\rightarrow$ check its $O(\uparrow)$
of both Matrix & List.

```
Array <bool> visited (grp.getSize());  
for (int i=0; i < grp.getSize(); i++)  
    visited[i] = false;
```

```
Queue <int> q;  
q.enqueue(sVertex);  
visited[sVertex] = true;  
while (!q.isEmpty())  
{
```

```
    int u = q.dequeue();
```

```
    cout << u << " ";
```

```
    for (int i=0; i < grp.getSize(); i++)  
    {
```

```
        if (!visited[i] && isEdge(u, i))  
        {
```

```
            visited[i] = true;
```

```
            q.enqueue(i);
```

```
        }
```

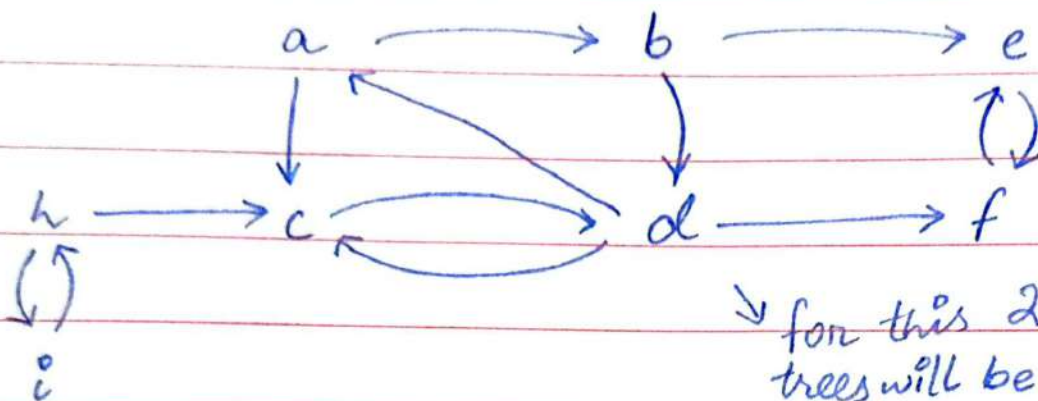
```
    }
```

```
}
```

```
}
```

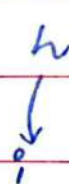
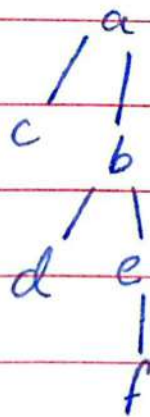
(Lecture no 824)

BFS Tree Visits:-



for this 2 Spanning trees will be formed.

↳ Start BFS at a and make a tree called spanning tree.



Multiple Spanning trees can be made for the same vertex. (Theoretically)

↳ BFS (int startVertex) → Graph g (grp.getSize());

```

{
    Array<bool> vs (grp.getSize());
    for (int i=0; i < vs.getSize(); i++)
        vs[i] = false;
    for (int i=0; i < vs.getSize(); i++)
        if (!vs[i])
            BFS(i, vs, g)
}
  
```

↳ Connected Graph:-

There is atleast one vertex through which whole graph can be traversed.

↳ Strongly Connected Graph:-

Whole graph can be traversed from any vertex as starting vertex.

↳ We can use counter in our BFS code to check if the graph is connected or not after BFS if the remaining vertices are $v-1$ then the graph is connected else it is not connected.

↳ DFS Function:-

DFS (int sVertex)

{ Graph g (g.grp.getSize());

Array <bool> vis;

for (int i=0; i < grp.getSize(); i++)
vis[i] = false;

~~11/11/2021~~ 11/11/2021

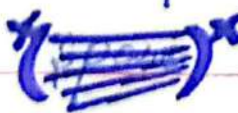
DFS (Vertex^{*}, vis, ^{graph}) → loop here to visit even the disconnected components.

```

↳ DFS (int u, Array <bool> & vis, graph by reference g)
{
    cout << u;
    vis[u] = true;
    for (v=0; v < gvp.getSize(); v++)
    {
        if (isEdge(u, v) && !vis[v])
        {
            DFS (v, vis);
        }
        g.addEdge(u, v);
    }
}

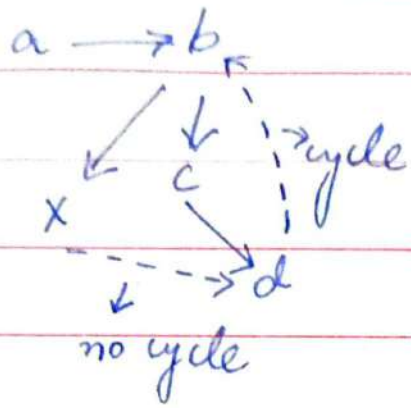
```

↳ Matrix → dense Graph.

↳ List → Sparse Graph. → Most Applications are  working with sparse data.

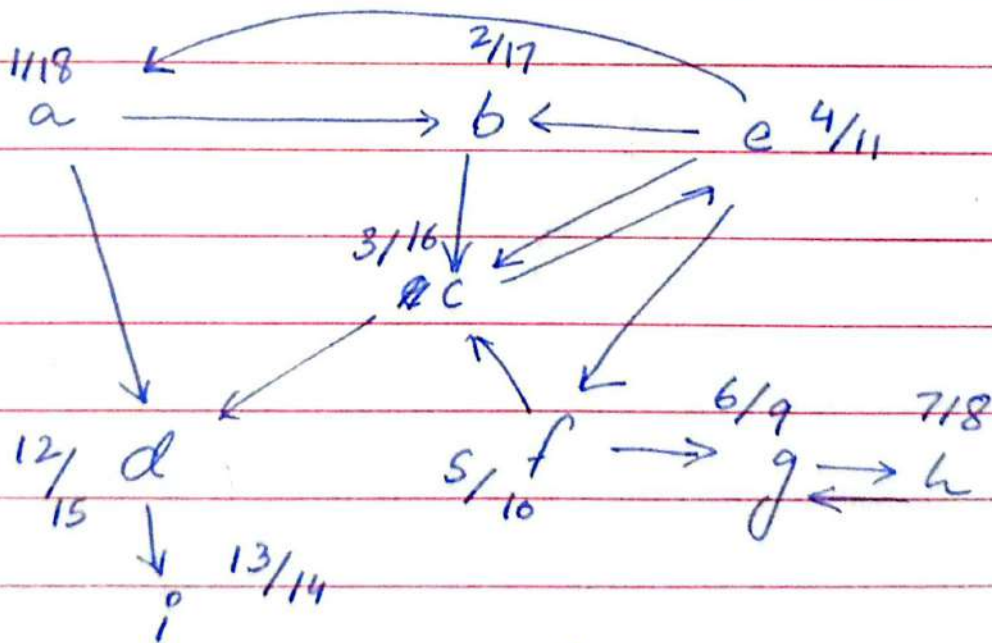
↳ Minimum Cost Spanning Tree :- Concept from Spanning Tree.

↳ To check cycle in directed graph.



∴ Have to do more than just checking visited.

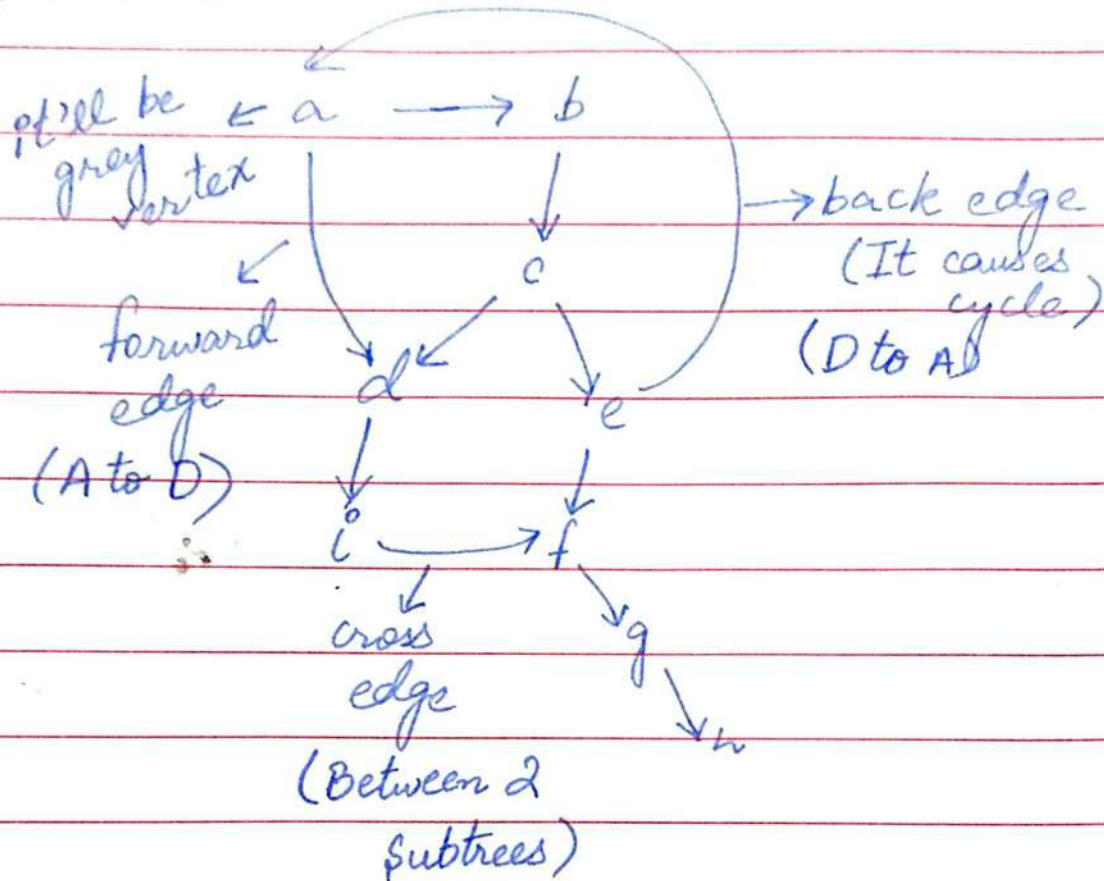
↳ We take two arrays i.e. (Start time & End time).



↳ enum COLOR $\{W, G, B\}$;

↳ Array <COLOR> status ; no of
↳ same size as ↑ vertices

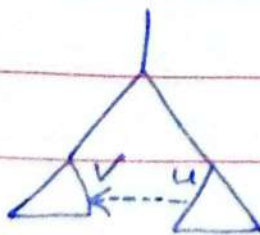
↳ Traversal Tree :-



• $u \rightarrow v$

_____ $u \rightarrow$ forward edge •
 _____ v

_____ $v \rightarrow$ back edge •
 _____ u



_____ v _____ u _____ $\rightarrow$ cross edge •

⇒ Strongly Connected in Undirected Graphs:-

↳ Check on vertex (DFS)

↳ Transpose then again check (DFS)

↳ This will be done in $O(V+V)$ cost.

• Strongly Connected Components (Sub Graph)

↳ Biconnected Components.

↳ Bipartite Graph.

Both of List & Matrix
Complexity of DFS and BFS (To do from home)
+ Spanning Algorithm cost Day

Date _____

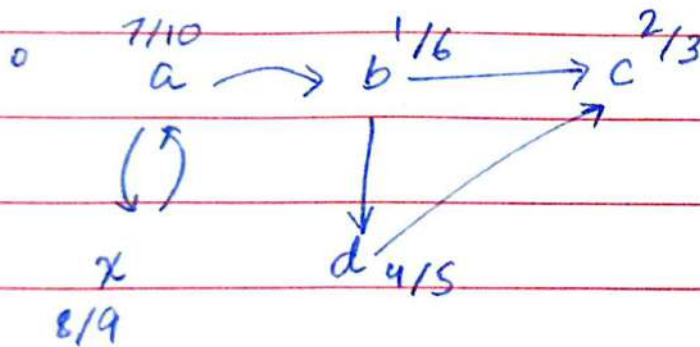
Day _____

Lecture no: 25

Strongly Connected Component :-

Can go to any vertex starting from any vertex.

↳ Node having largest end time isn't always strongly connected.



In order to get strongly connected components. Mark Time stamp then apply DFS on all nodes by their end time in Decreasing order. No. of DFS calls is the No. of connected components.

We'll use stack to get $\uparrow O(1)$ in DFS. We don't use MaxHeap because max finish time in

its insertion and removal cost will be $\log(n)$.

↳ It is not compulsory to store finish time.

↳ In undirected graph, only no. of DFS will be enough.

⇒ Minimum Cost Spanning Tree:-

↳ Algorithm for undirected graphs.

↳ Produces v vertices & $(v-1)$ edges.

↳ Prim's & Kruskal's Algorithm.

• Prime Algorithm:-

We'll make a minheap and keep the lowest weight edge out of all the nodes with the check that the edge isn't already between two visited vertices.

Date _____

Day _____

• Kruskal's Algorithm-

↳ Disjoint Set used.
↓
data structure.

∴ Cycle:- Within Same tree no ~~two~~ ~~vertices~~
~~two~~ ~~vertices~~ vertex produces
cycle.

~(DSA: Lecture no: 26)~

↳ Kruskal's Algorithm is done using disjoint set.

→ Disjoint Set
{

```
    Array<int> set;  
    public: t  
    bool doUnion(int u, int v);  
    int find(int v);  
    // constructors & destructors  
};
```

→ In Graph, two problem sizes are used V and E .
 ↓ ↓
 vertices Edges.

• BFS: /DFS

Matrix : $O(V^2)$ or $O(E)$ } more understandable
List : $O(V^2)$ or $O(E)$

• Shortest Path can be found out using DFS.

→ To find shortest path we'll pass on the weights and return when the value is ~~found~~ found.

⇒ Hashing:-

↳ Searching in $O(1)$.

↳ only ~~the~~ recommended to resolve equal queries.

↳ ordered map $\rightarrow$ tree.

↳ ordered map → tree.] → In STL.
↳ unordered-map → hashing.

o Key $\rightarrow$ Hash function $\rightarrow$ Hash Table

↓
output :
Hash Code

↓
Array

10

25

92

○

91

12

10

3425567

(low biasness)

prime k factors Ram
having main.

↳ Hash Collision:-

Same bucket address
for 2-3 keys by hash function.

• Hash Function mostly used :-

↳ Division Method :-

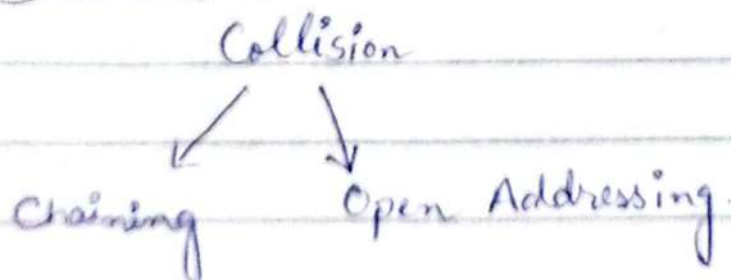
recommendation :-
should be prime.

$$\text{hash}(\text{key}) = \text{key} \% \text{tableSize}$$

↳ In the 26 size array for alphabet
counting it uses fixed data
i.e. (a-z) but it doesn't use
hashing

fixed data.

↳ Resolve Collision :-



↳ Load factor of hash table:-

$$\Rightarrow \frac{\text{no. of entries}}{\text{tableSize}}$$

⇒ ADT:-

```
class HashTable
{
    int tableSize = 31;
    Array<LSLL<int>> table;
    int entriesCount = 0;

public:
    HashTable(): table(tableSize)
    {}

    bool insert insert(int key)
    {
        int index = key % tableSize;
        table[index].insertAtHead(key);
        entriesCount++;
    }
    // Search if already exists
}
```

only storing key to store value use Pair(UD)

```
bool search(int key);  
✓ bool remove(int key);  
int hashCode(key);  
void rehash(); → make old array  
float III getLoadFactor(); after resizing copy  
}; again to store  
with newTableSize
```

→ Also make getNextPrime();

↳ prime number do better than
composite number but it doesn't
guarantee no collision.

~ (Lecture 8 nos 2.7) ~

↳ Division Method is used when you have very little knowledge about the keys.

↳ Range Queries $\Rightarrow$ Search Trees.

↳ Hashing $\Rightarrow$ equal Queries.

➤ Multiplication Method:-

$$\lfloor ((key * A) \bmod 1) * tableSize \rfloor$$

$$\downarrow$$
$$0.168 \Rightarrow \frac{\sqrt{5}-1}{2} \therefore \text{by Donald E. Knuth.}$$

$\downarrow$
it's a

recurrent number.

◦ Division Method is used because it is simple and fast. (Have only 1 operation).

* Go for the division method if you don't know what type of keys will come.

❖ Hashing for String:-

→ Polynomial Hash Code:-

$$a \times n^0 + a \times n^1 + \overset{a}{\cancel{13}} \times n^2$$

↓
31
↓
53 (Prime Numbers)

ASCII
codes.

❖ Folding:-

169 782 313 62

169

782

313

26

→ Hash Code.

❖ Open Addressing:- (All C++ libraries use this)

- ↳ Linear Probing.
 - ↳ Quadratic Probing.
 - ↳ Double Hashing.
- better performance

↳ Linear Probing:-

↳ Have to maintain a status array.

$$\hookrightarrow h(k, i) = h(k) + i, \text{ for } i = 1, 2, 3, \dots$$

$$\downarrow$$
$$(h(k) + i) \% \text{tableSize}$$

$\downarrow$
 $m \rightarrow$ in books
tho tableSize.

0	
1	c
2	e
3	a
4	b
5	a
6	a
7	d
8	f
9	

a $\rightarrow$ 3

e $\rightarrow$ 2

c $\rightarrow$ 1

f $\rightarrow$ 6

is Memory
Diagram.

→ To search if empty slot and still not found then return false. (issue will occur if it'll face ~~any~~ a deleted slot so we'll store Empty, Occupied and deleted. (enum SlotStatus))

↳ Quadratic Probing:-

In this the value of i (in linear Probing) will be :-
 $i^2 = 1^2, 2^2, 3^2, 4^2, \dots$

↳ The load factor of Quadratic Probing must be less than 50% for it to work properly.

∴ 2 Conditions :-

Guaranteed empty slots providing 2 conditions.

- 1) Load factor must be less than 0.50
- 2) tableSize should be prime.

key $\rightarrow 3$

0	
1	
2	e
3	a
4	b
5	
6	
7	d
8	
9	

Sequence:-

$$3 + 1^2 = 4$$

$$3 + 2^2 = 7$$

$$3 + 3^2 = 12 \rightarrow 2$$

(coprime logic)

To neglect

↑ 0 so jump can start.

$$K \% (m-1)$$

Double Hashing:-

Formula:-

$$h(k, i) = (h_1(k) + i * h_2(k)) \% m.$$

↓
→ $K \% \text{tableSize}$
2 hash functions.

$i = 1, 2, 3, 4, 5, 6, 7, \dots$

Any number less than a prime number is coprime of that number.

~ (Lecture 8 nos 28) ~

• Sorting:-

1. Selection Sort.

2. Bubble //

3. Insertion //

4. Heap //

5. Merge //

6. Quick //

7. Counting //

8. Radix //

↳ To find k th minimum element
then we would only
run the loop of bubble /
selection sort only k -times.

• Insertion Sort:-

↳ like we do
in real life.

↳ Best case: $O(n)$

↳ Worst Case: $O(n^2)$

↳ IS will perform better in BS because swap is heavy operation than shift.

Code:-

```
for (int i = 1; i < N; i++)  
{  
    val = arr[i];  
    int j = i - 1;  
    while (j >= 0 && val < arr[j])  
    {  
        arr[j+1] = arr[j];  
        j = j - 1;  
    }  
    arr[j+1] = val;  
}
```

Stable Algorithm:-

The relative order of equal elements should be same in the sorted Array/data.

10	5		10	5
20	4	→	30	5
30	5		20	4
			(Stable)	

↓

30	5
10	5

20 4

(Unstable)

↳

a - 10 - 10 - 10

10 - a - 10 - 10

what

if an a's
here

Counter Example [→] x_a
what if we don't
write $\leq$ inplace
of $<$ in Selection
Sort.

↳ Selection & Bubble Sort works
on fixed data whereas
Insertion sort can also
work on unfixed data/
array size.

~ (Lecture no: 29) ~

↳ Heap Sort

◦ `buildHeap(int arr[], int N);`

◦ `downHeap(int arr[], int N, int index);`

↳ Time = $\uparrow \log(n)$, Space = constant

↳ Max Heap for ascending.

↳ Min Heap for descending.

◦ Code

```
void heapSort(int *a, int N)
{
    buildHeap(a, N);
    int s = N;
```

```
    for (int i = 1; i < N; i++)
    {
```

```
        val = a[0];
```

```
        downHeap(a, s, 0);
```

```
        a[s-1] = val;
```

```
        s--;
```

```
    }
```

```
}
```

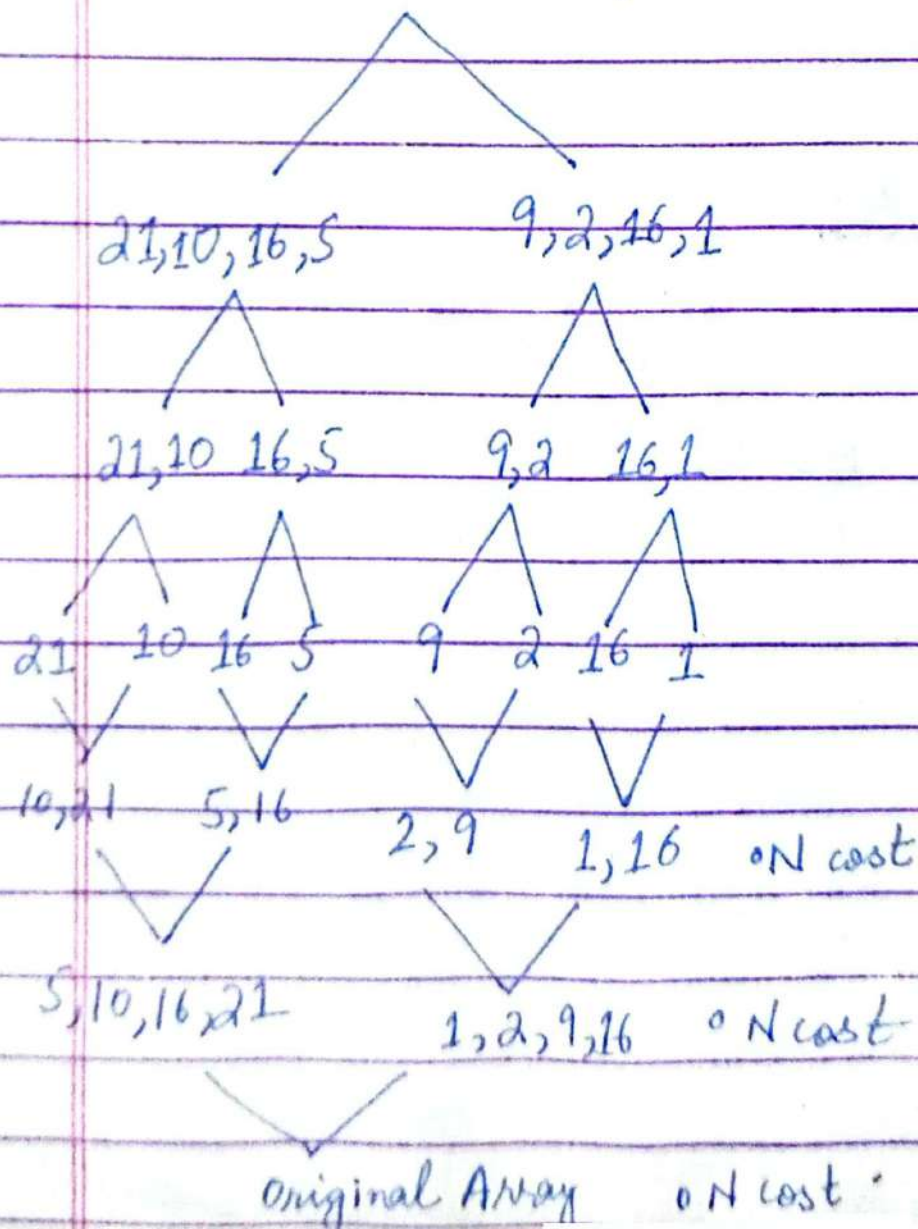
→ this removes last and places on the top.

↳ Merge Sort:-

↳ void merge(int *a, int aN, int *b, int bN);

• Abstracts-

21, 10, 16, 5, 9, 2, 16, 1



$\hookrightarrow$ Time Complexity = $O(n \log n)$,
 $\hookrightarrow$ Space Complexity = $O(1)$.

→ Code:-

```

void mergeSort (int *arr, int N)
{
    mergeSort (arr, 0, N-1);
}
  
```

```

void mergeSort (int *a, int l, int r)
{
    if (l == r)
        return;
    m = l + (r - l) / 2;
    mergeSort (a, l, m);
    mergeSort (a, m+1, r);
    merge (&a[l], m-l+1, &a[m+1],
           r-m, l-(l+1));
}
  
```

↳ Quick Sort:-

↳ Pivot Selection.

Code:-

```
int  
int qSortPos (int *a, int N,  
               int l, int r)  
{  
    int i = ll+1, j = rr;  
    while (i <= j) {  
        while (a[i] < a[l])  
            i++;  
        while (a[j] >= a[l])  
            j--;  
        if (i < j)  
            swap(a[i], a[j]);  
    }  
    swap(a[l], a[j]);  
    return j;  
}
```

↳ Time Complexity = $O(n \log n)$

↳ Space Complexity = $O(1)$

∴ Worst Case:-

1, 2, 3, 4, 5

(Ascending Order)

↳ Improvement: find pivot
more randomly.

↳ Improved versions will always
outperform other algorithms.

Code:-

```
void qQuickSort (int* arr, int N)
{
    quickSort(arr, 0, N-1);
}

void quickSort (int* a, int l, int u)
{
    if (l == u)
        return;
    int x = qSortPos(a, l, u);
    quickSort(a, l, x-1);
    quickSort(a, x+1, u);
}
```

↳ All we studied now are
comparison based sorting.

↳ Counting Sort:-

↳ Non-Comparison based
Sorting.

↳ Works on small integers.

Abstract:-

10, 2, 6, 6, 7, 3, 3, 6, 1, 2, 4, 5, 5,
8, 9, 8

0	
1	1
2	2
3	2
4	1
5	2
6	3
7	1
8	2
9	1
10	1

∴ If stability
isn't required
than this
version is
fine.

Array <LSLL> Version:-

0	
1	→ 1
2	→ 2 → 2
3	→ 3 → 3
4	→ 4
5	→ 5 → 5
6	→ 6 → 6 → 6
7	→ 7
8	→ 8 → 8
9	→ 1
10	→ 10

More Refined Version:-

0	0	Prefix Sum → Array	0	0
1	1		1	1
2	2		3	2
3	2		5	3
4	1		6	4
5	2		8	5
6	3		11	6
7	1		12	7
8	2		14	8
9	1		15	9
10	1		16	10

↳ Max Value Array:-

0	1	↳ Reverse Loop Array	
1	2	0	
2	2	1	
3	3	2	
4	3	3	
5	4	4	
6	5	5	
7	5	6	
8	6	7	
9	6	8	
10	6	9	
11	7	10	
12	8	11	
13	msm	12	8
14	9	13	8
15	10	14	9
		15	

↳ As we start filling we'll
minus count from prefix
sum.

↳ Radix Sort:-

↳ Stability is imp.

Ex.

6, 31, 10, 21, 103, 1005, 21, 21, 36,

2

0	→ 10	% wrt unit digit.
1	→ 31, 21, 21	
2	→ 2	
3	→ 103	
4		
5		
6	→ 6, 36	
7		
8		
9	→ 29	

↓
because its base 10 numbers (0-9)

↳ After pass 1:

10, 31, 21, 21, 2, 103,
1005, 6, 36, 29.

0

↳ After Pass-2:-

2, 103, 1005, 6, 10, 21, 21, 29,
31, 36.

↳ After pass-3:-

2, 1005, 6, 10, 21, 21, 29, 31, 36,
103

↳ After pass-4:-

2, 6, 10, 21, 21, 29, 31, 36,
103, 1005.